Advanced React Development

Modern React for 2026 — and the developers who use it

Module 1: Welcome + The 2026 React Landscape

Chris Minnick · WatzThis · v2.1 · May 2026

Welcome

Three days. Eight labs. A lot of new React.

Your instructor

Chris Minnick

Founder, WatzThis

25+ years building for the web

Author: JavaScript All-In-One For Dummies, Beginning ReactJS Foundations, others

Teaching React since 2015

And you?

- Your name and role
- How long you've worked with React
- What you're building right now
- One thing you want to leave this class knowing

Why this course was rewritten

The 2024 version is barely two years old, but React has moved a lot.

React 19 shipped

Server Components production-ready, the Actions story, and a half-dozen new hooks worth knowing.

The React Compiler is real

Most of the manual useMemo / useCallback / React.memo gymnastics from 2024 is now optional.

Create React App is gone

Officially deprecated in 2025. Vite is the default; frameworks own the rest.

AI assistants are in every editor

The 2024 course didn't mention them. The 2026 course is built around the skills you need to use them well.

What you'll be able to do by Friday

Ten course-level outcomes. The labs are how you get there.

1. Recognize what's new in React 19 and explain when each piece matters
2. Choose between Vite, Next.js 15, React Router v7, and the rest based on project needs
3. Apply the right hook for the job — and spot the hook mistakes AI tools make
4. Implement modern routing in both React Router v7 and Next.js App Router
5. Pick the right state-management approach for each kind of state
6. Build with TanStack Query for client data and Server Components for server-driven UI
7. Profile and optimize a React app in the React Compiler era
8. Write meaningful tests with Vitest, React Testing Library, MSW, and Playwright
9. Work productively with AI assistants while spotting their typical React mistakes
10. Apply React-specific security and accessibility practices

Daily schedule

~7 hours of instruction per day. Lunch is one hour. Two breaks each day.

Day 1

Foundations for 2026

- Modern React landscape
- Hooks that matter in 2026
- Routing — RR v7 + Next.js
- Labs 1 & 2

Day 2

Data, State, and the Server

- State management decisions
- Modern data fetching
- Server Components + Actions
- Labs 3, 4, & 5

Day 3

Performance, Quality, and AI

- Performance in the Compiler era
- Testing in 2026
- Working with AI assistants
- Labs 6, 7, & 8

How the days are structured

Concept → quick demo → substantial lab. Repeat.

Concept

Short lecture (15-30 min) with live coding, not just slides.

Demo

5-10 min hands-on demo so the syntax lands before you write any.

Lab

60-90 min hands-on. Every lab has a stretch task for fast finishers.

51% hands-on labs across the course

The lab repo

Clone it now if you haven't already.

github.com/chrisminnick/advanced-react-19

Version 2.0 · Lab starters in lab-files/lab-01 .. lab-08 · Reference solutions in solutions/

Starter projects

Per-lab folders in lab-files/lab-01 through lab-08 — each fully self-contained

Reference solutions

One folder per lab under solutions/ — sometimes more than one when there's a pick-your-tool moment

Per-project READMEs

Setup, run instructions, and what each lab folder contains

AI assistants in this course

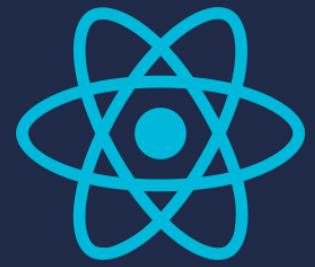
Tool-agnostic. Use what you use.

What we're doing

- Treating AI as a real part of the workflow — every day
- Building review skills: spotting what AI gets wrong about React
- A focused module on Day 3 about working with AI well
- Comparing tools when it's instructive, but not picking favorites

What we're not doing

- Banning AI in labs — that would be teaching to a 2022 reality
- Required tooling — bring Cursor, Copilot, Claude Code, or whatever
- Treating AI output as authoritative — we'll review every line
- Pretending AI replaces understanding the framework



Part 1

The 2026 React Landscape

React 19, the Compiler, frameworks, and the AI layer on top.

Where we left off — React 18

Released March 2022. The era most production code is still written for.

Concurrent rendering

Suspense for code splitting, useTransition, useDeferredValue

Automatic batching

Promise callbacks, timeouts, and native handlers all batch updates

Streaming SSR

renderToPipeableStream — but you wired everything yourself

useId, useSyncExternalStore

Hydration-safe IDs and clean integration with non-React state

React 19 — what shipped, when

From experimental Server Components to a stable platform.

React 19 is the first release with a coherent server story baked in. That's the headline.

What's new in React 19

We'll touch each of these today. Hooks deep dive is next module.

Actions

Functions you pass to <form> action props. Mutations + pending state without ceremony.

New hooks

use(), useActionState, useFormStatus, useOptimistic — covered in Module 2.

ref as a prop

forwardRef is no longer required. Pass ref like any other prop.

Server Components

Stable. Component code that runs only on the server. Module 6 covers this.

Asset preloading

preload, preinit, preconnect — fine-grained control over what loads when.

Document metadata

<title> and <meta> tags work as plain JSX inside components.

Actions: forms without the wiring

Pass a function to <form action>. React handles pending, errors, and resets.

```
function NewPostForm() {
  async function createPost(formData) {
    await savePost(formData.get("body"));
  }

  return (
    <form action={createPost}>
      <textarea name="body" />
      <SubmitButton />
    </form>
  );
}
```

What you don't write anymore:

- isPending state — React tracks it (use useFormStatus inside)
- preventDefault — Actions intercept submit
- Manual reset — form clears on success
- Race-condition handling — newer submissions win

Pair with useActionState for return values, errors, and validation. Module 2.

Four hooks worth circling

We'll use each of them in Lab 1. Coverage in Module 2.

`use(promise)`	Suspends until a promise resolves. The new way to read async data.
`useActionState`	Pairs with Actions. Gives you state, the action, and a pending flag.
`useFormStatus`	Read pending/data/method from inside a child of <form>. No prop drilling.
`useOptimistic`	Show the eventual state immediately. Reverts on error.

ref as a prop

forwardRef is gone. ref behaves like any other prop in React 19.

Before — React 18

```
const Input = forwardRef(
  (props, ref) => (
    <input ref={ref} ...props />
  )
);
```

After — React 19

```
function Input({ ref, ...props }) {
  return <input ref={ref} {...props} />;
}
```

Codemod available — but legacy forwardRef still works through a deprecation warning. Plan, don't panic.

Server Components — production-ready

Components that run on the server only. Zero JS shipped to the browser.

Server Component

- Runs on the server only
- Can fetch directly — DB, file system, API
- Imports stay on the server
- No useState, useEffect, or browser APIs

Client Component

- Marked with 'use client' directive
- Hydrates in the browser
- Interactive — handlers, state, effects
- Receives props (incl. children) from Server Components

Module 6 covers Server Components in depth, including the boundary mistakes AI tools love to make.

Two smaller wins worth knowing

Asset preloading and document metadata are now first-class in React 19.

Asset preloading

```
preload(url, options)
preinit(url, options)
preconnect(url)
```

Move resource-loading hints into the components that actually need them — no more <Helmet> or framework-specific shenanigans.

Document metadata

```
<title>About</title>
<meta name="description" ... />
```

Drop title and meta tags inline anywhere in your component tree. React deduplicates and hoists.

The React Compiler

Auto-memoization for components and hooks. Most useMemo/useCallback becomes optional.

Before — manual memo

```
const filtered = useMemo(
  () => items.filter(f), [items, f]
);
const onClick = useCallback(
  (e) => save(e), [save]
);
export default memo(MyList);
```

After — Compiler enabled

```
const filtered = items.filter(f);
const onClick = (e) => save(e);

export default MyList;

// Compiler memoizes
```

Enabled per-project. Pair with the eslint plugin so it tells you which components opted out.

What the Compiler doesn't do for you

Knowing the limits is what keeps performance work honest.

Doesn't fix Rules of Hooks violations

Same rules as before. If anything, the eslint plugin gets stricter — opt-in violations bail out of memoization.

Doesn't memoize across mutation

Mutating an object and reusing it still breaks memoization. Use immutable updates.

Doesn't replace algorithmic optimization

If your filter is $O(n^2)$, the Compiler can't save you. Look at the data, not just the renders.

Doesn't help with non-React work

A blocking 200ms request is still a blocking 200ms request. Web Workers, Suspense, and streaming still apply.

Part 2

Bootstrapping in 2026

Vite, frameworks, and choosing not to choose a framework.

Create React App is gone

Officially deprecated by the React team in February 2025.

`create-react-app` → **deprecated**

Why now:

- Webpack-based build couldn't keep up with modern dev-server expectations (HMR, ESM)
- React 19's server-driven features need a framework or a Vite-style native build
- Maintenance burden on the React team without strategic upside
- Existing CRA projects keep working — but new projects shouldn't start there

Vite is the new default

If you're not using a framework, you're probably using Vite.

```
$ npm create vite@latest

✓ Project name … my-app
✓ Framework … React
✓ Variant … JavaScript
```

What you get:

- Native ESM dev server (instant cold start)
- Rollup-based production builds
- Sensible HMR out of the box
- JS, TS, JSX, TSX support without config
- Plugin ecosystem covers everything CRA used to plus React Router

Add the React Compiler with babel-plugin-react-compiler in your Vite config. The eslint plugin tells you which components opted out.

When plain Vite + React is enough

Don't reach for a framework reflexively. Sometimes Vite is the right answer.

- Internal tools and dashboards behind auth — SEO doesn't matter, server rendering doesn't matter
- Single-page applications where the data model is mostly client-side state
- Embeddable widgets shipped as a single bundle into a host page
- Prototypes and proofs-of-concept where shipping speed beats architectural perfection
- Storybooks, design systems, component libraries — not really apps in the framework sense

When you do need a framework

Most consumer-facing apps land here.

- Public-facing pages that need SEO and shareable URLs
- Server Components and Server Actions — they require a framework runtime
- Streaming SSR with sensible defaults (the renderToPipeableStream-by-hand path is a sharp edge)
- File-based routing, route loaders, and built-in data fetching conventions
- Production deployment story: edge runtime, ISR, partial prerendering

Framework picks in 2026

We'll teach in two of these. The others are useful to recognize.

Next.js 15	Vercel-led. App Router + Server Components + Server Actions. Largest ecosystem.	taught
React Router v7 (framework)	Merged with Remix in late 2024. Loaders, actions, framework mode. Strong fit for app-style sites.	taught
TanStack Start	Built on TanStack Router + TanStack Query. Newer; appealing if you live in TanStack land.	
Astro	Content-first. React as one of many island flavors. Great for marketing sites and docs.	

Lab 2 implements the same auth flow in both Next.js 15 and React Router v7 framework mode. You'll see them side-by-side.

Next.js 15

Most popular React framework. App Router + RSC by default in new projects.

Strengths

- First-class Server Components and Server Actions
- Streaming + Suspense baked into the rendering path
- Partial prerendering for hybrid static/dynamic pages
- Turbopack stable for dev — fast builds
- Vercel deployment story is genuinely good

Watch-outs

- App Router conventions are non-trivial — file-as-API takes getting used to
- Caching defaults change between minor versions; read the release notes
- Vercel-flavored runtime — works elsewhere but the polish is uneven
- Easy to over-reach for server features when a Vite SPA would do

React Router v7 — framework mode

RR + Remix merged in November 2024. One product, two modes.

Library mode = the React Router you know. Framework mode = Remix-style loaders, actions, and conventions, with Vite under the hood.

Strengths

- Closer to vanilla React than Next.js — fewer conventions to memorize
- Loaders + actions = a clean data-on-the-server story
- Deploy anywhere Node runs — no vendor pull

Watch-outs

- Smaller ecosystem of third-party RSC integrations than Next.js
- Less material online — search results still mix RR v6 and Remix v2 advice
- Server Components support is solid but newer than Next.js's

Worth knowing about

Not what we'll teach, but worth knowing the shape of.

TanStack Start

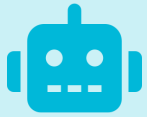
Newer entrant. Built on TanStack Router + TanStack Query. Type-safe, file-based routing, server functions. Worth a look if you're already TanStack-heavy or you want type safety as a first-class feature.

Astro

Content-first framework. React renders as 'islands' inside otherwise-static pages. Best for marketing sites, docs, and content-heavy apps where most pages don't need much JS at all.

AI assistants are part of the workflow now

And React is one of the things they get most-frequently-wrong.



Most students in this room used an AI assistant to help with at least one React component this week.

React-specific failure modes you'll see across all the major tools:

- Stale closures inside useEffect callbacks
- Missing or over-eager dependency arrays
- useEffect for derived state (should be plain calculation)
- Server/client boundary leaks (importing server code into 'use client')
- Outdated React Router APIs from pre-v6.4 era
- Hydration mismatches from non-deterministic renders

Coming up next

Module 2 — Hooks That Matter in 2026.

Module 2 — Hooks That Matter

75 minutes · Lecture + live demos

In the next module:

- The hooks rulebook — and why AI assistants violate it constantly
- When not to reach for useEffect
- use(), useActionState, useFormStatus, useOptimistic — with real code
- useSyncExternalStore, useDeferredValue, useTransition, useLayoutEffect
- Then Lab 1: modernize the chat app — your first AI-review exercise

Module 2

Hooks That Matter

Use the right hook for the job. Spot the wrong one in AI-generated code.

75 minutes · Lecture + live demos · Lab 1 follows

How many hooks does React 19 have?

Twenty-ish. We won't cover them all. We'll cover the ones that earn their keep.

≈ 20

built-in hooks in React 19

4 are new since React 18

Plus ~6 framework- or compiler-only hooks not covered today

In this module:

- The hooks rulebook (and how AI breaks it)
- Foundational hooks revisited — useState, useReducer, useEffect (and when not to)
- The four new arrivals — use, useActionState, useFormStatus, useOptimistic
- Specialty hooks — useSyncExternalStore, useDeferredValue, useTransition, useLayoutEffect
- Then Lab 1: modernize the chat app

Part 1

The Rules of Hooks

Five minutes. AI assistants violate these constantly — knowing them is part of the review skill.

The Rules of Hooks

Three rules. The eslint plugin enforces them. Most AI tools eventually break them anyway.

1

Only call hooks at the top level

Never inside loops, conditions, nested functions, or after early returns. React tracks hooks by call order; conditional calls break that order.

2

Only call hooks from React functions

Function components, custom hooks (functions that start with use). Not regular functions, not class methods, not event handlers.

3

Custom hooks compose by calling other hooks

If you find yourself wanting to call a hook conditionally, lift the conditionality into the component using the custom hook.

Why the rules exist

React tracks hook state by call order, not by name. Conditionals would scramble it.

Breaks the rules

```
function Greeting({ name }) {
  if (!name) return null;
  // Hook called only when name truthy
  const [count, setCount] = useState(0);
  return <h1>Hello {name}</h1>;
}
```

Plays by the rules

```
function Greeting({ name }) {
  // Hook called every render
  const [count, setCount] = useState(0);
  if (!name) return null;
  return <h1>Hello {name}</h1>;
}
```

React's reconciler relies on hooks being called in the same order every render. Break the order, get a confusing 'Rendered fewer hooks than expected' error.

AI assistants violate these constantly

Why review skill matters more than typing skill.

Hook inside a conditional

Asked to 'only run this when X', AI tools wrap a useEffect in if (X) {} more often than they reach for the dependency array.

Hook in a callback

useState inside an onClick handler is depressingly common in AI-generated code.

Hook ordering changes between renders

Different hook called depending on a prop value — silently breaks state across re-renders.

Custom hook called conditionally

When the AI is told 'use this when…', it produces useMyThing() inside an if. Same problem.

Part 2

Foundational Hooks Revisited

useState, useReducer, useEffect — and the situations where useEffect is the wrong tool.

useState — what most people know

Three patterns worth being deliberate about.

Functional updater

When new state depends on previous state — always.

```
// race-prone
setCount(count + 1);

// safe
setCount(n => n + 1);
```

Lazy initialization

When the initial value is expensive to compute.

```
// runs every render
useState(expensive());

// runs once
useState(() => expensive());
```

State updater batching

React 19 batches all updates in event handlers, effects, and timers automatically.

```
// one render, count = 3
function onClick() {
  setCount(n => n + 1);
  setCount(n => n + 1);
  setCount(n => n + 1);
}
```

useState trap: derived state

If you can compute it from props, don't put it in state.

The trap

```
function List({ items, query }) {
  const [filtered, setFiltered] = useState(items);

  // bug: filtered lags items by one render
  useEffect(() => {
    setFiltered(items.filter(i =>
i.includes(query)));
  }, [items, query]);

  return <ul>{filtered.map(...)}</ul>;
}
```

The fix

```
function List({ items, query }) {
  // just compute it
  const filtered = items.filter(
    i => i.includes(query)
  );

  return <ul>{filtered.map(...)}</ul>;
}

// Compiler memoizes if needed
```

This is the single most common AI-generated React mistake. State + useEffect to track props is almost always wrong.

useReducer — when state has multiple actions

Better than useState when transitions are complex or related states change together.

Use useReducer when:

- Multiple state values change together
- State transitions are non-trivial (validation, side-effect dispatch)
- You want to test the transition logic in isolation
- You're modeling a state machine

```
function reducer(state, action) {
  switch (action.type) {
    case 'add':
      return { ...state,
        items: [...state.items, action.item]
      };
    case 'reset':
      return initialState;
  }
}

const [state, dispatch] = useReducer(
  reducer, initialState
);
```

TS tip: action.type as a discriminated union gives you exhaustive checks.

useEffect — what it's actually for

Synchronizing your component with something outside React.

Yes — synchronize external systems

- Subscribing to an external store (event bus, websocket, browser API)
- Setting up timers or animation frames
- Imperatively mutating a third-party widget (a chart, a map, a video player)
- Logging to analytics on mount or unmount

No — these are not effects

- Computing values you can derive from props or state
- Resetting state when props change (use a key prop instead)
- Fetching data (use Suspense + use(), or TanStack Query)
- Triggering events in response to user actions (use event handlers)

When not to useEffect

Four patterns that look like effects but aren't. Use these instead.

useEffect to compute derived data	Compute it inline in render
useEffect to reset state on prop change	Pass a key prop
useEffect to fetch data	Suspense + use() or TanStack Query
useEffect to dispatch on user action	Move the dispatch into the event handler

Stale closures

The bug eslint catches and AI tools cause.

The bug

```
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const id = setInterval(() => {
      // count is always 0 — stuck at 1!
      setCount(count + 1);
    }, 1000);
    return () => clearInterval(id);
  }, []); // count is captured as 0
}
```

The fix

```
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const id = setInterval(() => {
      // reads current count
      setCount(n => n + 1);
      console.log(count);
    }, 1000);
    return () => clearInterval(id);
  }, []); // no dep needed
}
```

The dependency array rules

react-hooks/exhaustive-deps catches almost all of these. Don't disable it.

Every reactive value used inside the effect must be in the array
props, state, context, derived values from those — not refs (refs don't trigger re-renders)

Don't disable the lint rule to silence warnings
If exhaustive-deps complains, the code is probably wrong. Fix the underlying issue.

Object/function dependencies will re-run the effect every render
Memoize them, or move them inside the effect, or refactor to avoid the dependency.

Empty array means 'on mount/unmount only'
If you want that, double-check that nothing reactive is captured. Consider whether you really need an effect at all.

Part 3

The Four New Arrivals

use, useActionState, useFormStatus, useOptimistic — the React 19 form & async story.

The four hooks worth circling

These didn't exist in the 2024 course. They will be in your reviewer's mental model now.

`use(promise | context)`	Read async values inline. Suspends the component until the promise resolves.
`useActionState`	Pairs with form Actions. Gives you state, the wrapped action, and a pending flag.
`useFormStatus`	Read pending/data/method from inside any descendant of a <form>. No prop drilling.
`useOptimistic`	Show the eventual state immediately. React reverts on error.

use() — read promises and contexts

The first hook that's allowed inside conditionals. Suspends instead of returning a value.

Two acceptable inputs:

`use(promise)`

Suspends the component until the promise resolves. Pair with a <Suspense> boundary above.

```
function Profile({ userPromise }) {
  const user = use(userPromise);
  return <h1>{user.name}</h1>;
}
```

`use(context)`

Like useContext, but legal inside if/loop/early-return. Quietly the unsung benefit.

```
function Header() {
  if (!hasAuth) return null;
  const theme = use(ThemeContext);
  return <nav style={theme.nav} />;
}
```

use(promise) in practice

Pair with <Suspense> above. Don't create the promise inside the component that calls use().

```
// page.jsx — Server Component
function ProfilePage({ params }) {
  // fetch starts here, but we don't await
  const userPromise = fetchUser(params.id);
  return (
    <Suspense fallback={<Spinner />}>
      <Profile userPromise={userPromise} />
    </Suspense>
  );
}

// Profile.jsx — Client Component
'use client';
export function Profile({ userPromise }) {
  const user = use(userPromise);
  return <h1>{user.name}</h1>;
}
```

Common mistake: creating the promise inside the component that calls use(). New promise every render → infinite suspense loop.

useActionState — anatomy

Wraps a form action. Gives you state, the action, and isPending in one tuple.

```
const [state, action, isPending] = useActionState(
  fn,        // async (prevState, formData) => newState
  initialState,
  permalink, // optional, for progressive enhancement
);
```

state

Whatever fn returned last time it ran. The classic form-state pattern: { errors, fieldValues, success }.

action

A wrapped version of fn. Pass it to <form action>. React calls it on submit and updates state automatically.

isPending

True while the wrapped action is running. Use for spinners and disabling submit.

useActionState in practice

Login form with validation, error display, and a pending state — no useState needed.

```
async function login(prevState, formData) {
  const email = formData.get('email');
  const password = formData.get('password');
  const result = await authApi.login(email, password);
  if (result.error) return { email, error: result.error };
  redirect('/home');
}

function LoginForm() {
  const [state, action, isPending] = useActionState(login, {});
  return (
    <form action={action}>
      <input name='email' defaultValue={state.email} />
      <input name='password' type='password' />
      {state.error && <p className='error'>{state.error}</p>}
      <button disabled={isPending}>Sign in</button>
    </form>
  );
}
```

useFormStatus — the inside-form hook

Read pending/data/method/action from any descendant of a form. Replaces prop drilling.

```
const { pending, data, method, action } =
  useFormStatus();
```

Returns:

- pending — true while the parent form's action is running
- data — the FormData being submitted (or null)
- method — 'get' or 'post'
- action — the function the form will run

Must be called from inside a component rendered as a descendant of <form>. Calling it on the form itself returns null.

useFormStatus in practice

A submit button that knows when its form is submitting — without any props.

```
// reusable, no props needed
function SubmitButton({ children }) {
  const { pending } = useFormStatus();
  return (
    <button type='submit' disabled={pending}>
      {pending ? 'Saving…' : children}
    </button>
  );
}

// elsewhere
<form action={savePost}>
  <textarea name='body' />
  <SubmitButton>Post</SubmitButton>
</form>
```

useOptimistic — show the future, revert on error

Render the eventual state immediately. React reconciles when the real action completes.

```
const [optimisticState, addOptimistic] = useOptimistic(
  state,              // the 'real' state
  (currentState, optimisticValue) => newState
);
```

Use when…

User actions that have a small chance of failing — sending a message, liking a post, marking a todo done.

Don't use when…

Actions that are slow or likely to fail (uploads, payments). The reversal feels worse than just showing pending.

Pair with…

A Server Action or async handler. The optimistic state is replaced when the real state updates.

useOptimistic in practice

Send a message; it appears in the list instantly. If the server rejects it, it disappears.

```
function Thread({ messages, send }) {
  const [optimistic, addOptimistic] = useOptimistic(
    messages,
    (current, newMsg) => [...current, { ...newMsg, sending: true }]
  );

  async function submit(formData) {
    const text = formData.get('text');
    addOptimistic({ text });          // instant
    await send(text);                  // real call
  }

  return (
    <form action={submit}>
      {optimistic.map(m => <Bubble msg={m} />)}
      <input name='text' /><button>Send</button>
    </form>
  );
}
```

Putting them together

A complete posting form using all four new hooks. This is the pattern to remember.

```
function Composer({ posts }) {
  const [state, action, isPending] = useActionState(createPost, { ok: true });
  const [optimisticPosts, addOptimistic] = useOptimistic(
    posts,
    (current, newPost) => [{ ...newPost, sending: true }, ...current]
  );

  return (
    <>
      <form action={async (fd) => {
        addOptimistic({ body: fd.get('body') });
        await action(fd);
      }}>
        <textarea name='body' />
        <SubmitButton />            // uses useFormStatus
        {state.error && <Error />}
      </form>
      <PostList posts={optimisticPosts} />
    </>
  );
}
```

Part 4

Specialty Hooks

useSyncExternalStore, useDeferredValue, useTransition, useLayoutEffect — when each one earns its keep.

useSyncExternalStore

Subscribe to a non-React store safely — without tearing during concurrent renders.

Use when:

- Wrapping a Zustand/Redux/Jotai-style store you wrote
- Reading window.matchMedia, navigator.online, etc.
- Wrapping a third-party event emitter
- Anywhere the value can change outside of React

```
function useOnline() {
  return useSyncExternalStore(
    (notify) => {
      window.addEventListener('online', notify);
      window.addEventListener('offline', notify);
      return () => { /* cleanup */ };
    },
    () => navigator.onLine,        // client
    () => true                          // server
  );
}
```

useDeferredValue

Show stale content during expensive renders. Like a built-in stale-while-revalidate for state.

```
function SearchPage() {
  const [query, setQuery] = useState('');
  const deferred = useDeferredValue(query);

  return (
    <>
      <input value={query}
        onChange={(e) => setQuery(e.target.value)} />
      <Results query={deferred} />
    </>
  );
}
```

What you get:

- Input stays responsive — query updates immediately
- Results may lag a render or two — deferred value catches up
- Pair with React.memo or useMemo on Results ONLY if the React Compiler bails out
- Useful when you can't move the work to a worker

useTransition

Mark a state update as non-blocking. Updates that would freeze the UI become interruptible.

```
function Tabs() {
  const [tab, setTab] = useState('home');
  const [isPending, startTransition] = useTransition();

  function selectTab(next) {
    startTransition(() => setTab(next));
  }

  return <Layout busy={isPending}><Pane tab={tab} /></Layout>;
}
```

useDeferredValue vs useTransition

They solve adjacent problems. Pick by what you control.

`useDeferredValue`

Wrap a value you receive (props or state) so consumers see a delayed copy.

- Use when you don't own the setter — e.g., wrapping query from props
- The deferred value silently lags during expensive renders
- No isPending flag (compare deferred === actual instead)

`useTransition`

Wrap a setter call so the resulting work is interruptible.

- Use when you own the setter — wrap setX in startTransition
- Returns isPending — surface it as a spinner or busy indicator
- Newer transitions interrupt older ones automatically

useLayoutEffect

Like useEffect, but synchronous — fires before the browser paints. Use it sparingly.

Use when:

- You need to measure DOM and adjust layout before paint (e.g., a tooltip's position)
- You need to sync DOM mutations across reads/writes that would flicker otherwise
- You're writing a component library that has to interop with imperative DOM code

Don't use when:

- Plain useEffect would do — it blocks paint, hurting perceived performance
- Subscribing to data — neither hook should fetch
- Server rendering — useLayoutEffect doesn't run on the server, gets noisy warnings

Rule of thumb: reach for useEffect first. If you see a flicker between paint and effect, then upgrade to useLayoutEffect.

Hook selection cheat sheet

When in doubt: which hook fits the situation?

Local component value that changes	`useState`
Multiple related state values, complex transitions	`useReducer`
Read async data inline (Server Components)	`use(promise)`
Read context, possibly conditionally	`use(context)`
Form action + state + pending	`useActionState`
Submit button needing form pending state	`useFormStatus`
Show eventual state immediately	`useOptimistic`
Subscribe to non-React store	`useSyncExternalStore`
Keep input responsive during expensive renders	`useDeferredValue`
Make a state update interruptible	`useTransition`
Sync to an external system on render	`useEffect`
Measure DOM before paint	`useLayoutEffect`

AI hook mistakes — your review checklist

What to look for when reviewing AI-generated React.

- useState for derived data that should be a plain calculation
- useEffect to reset state on prop change (use a key prop instead)
- useEffect to fetch data (use Suspense + use, or TanStack Query)
- Stale closure in useEffect (missing dep, or use functional setState)
- Hook inside a conditional (lift the conditionality outside)
- Custom hook called conditionally (lift the conditionality)

Coming up: Lab 1

Modernize a legacy class-component chat app — your first AI-review exercise.

Lab 1: Modernize the Chat App

90 minutes · Hands-on · Stretch task: add useOptimistic

What you'll do:

- Open lab-files/lab-01/real-time-chat — class components, React 18, CRA
- Use an AI assistant to do a first-pass conversion to function components and modern hooks
- Review what the AI produced — spot at least three mistakes from the checklist
- End with a working modernized chat app and a short written review of the AI's mistakes
- Stretch: add useOptimistic so messages appear instantly while the round-trip completes

Module 3

Routing

React Router v7 framework mode and Next.js App Router, side by side.

60 minutes · Lecture + side-by-side demos · Lab 2 follows

The routing landscape

Three serious choices. We'll teach two and recognize the third.

React Router v7
Library mode + framework mode (Remix merged)

taught in this module

Next.js 15 App Router
File-system routing + Server Components + Actions

taught in this module

TanStack Router
Type-safe, file-based, headless. Newer; growing fast.

React Router v6 still works, but new projects start with v7. Old code with BrowserRouter / Routes / Route is still v7-compatible — just no longer the default starter.

Part 1

React Router v7

Library mode for SPAs. Framework mode for the Remix-style stack. One product, two front doors.

The November 2024 merger

Remix didn't disappear. It became the framework mode of React Router v7.

What happened

- React Router and Remix had drifted into overlap
- Both used loaders, actions, nested routes, error boundaries
- Remix Run shipped its merge plan in 2024
- RR v7 = the merged product, released Nov 2024
- Migration codemod handles most v6 → v7 changes

Two modes

Library mode

The classic React Router. Routes in JSX or createBrowserRouter. You bring your own bundler, dev server, and SSR.

Framework mode

The Remix-style stack. File-based routes, Vite under the hood, server runtime included. Loaders + actions are first-class.

Library mode — when it fits

Bring-your-own bundler. Just routing inside an existing app.

Reach for library mode when:

- You already have a Vite SPA
- You're embedding routing in a non-React shell
- Server rendering doesn't matter
- You want the smallest dependency surface

```
import { createBrowserRouter,
  RouterProvider } from 'react-router';

const router = createBrowserRouter([
  { path: '/', element: <Home /> },
  { path: '/about', element: <About /> },
]);

function App() {
  return <RouterProvider router={router}
/>;
}
```

Framework mode — getting started

One command. File-based routes. Loaders, actions, and SSR built in.

```
$ npx create-react-router@latest my-app
```

What you get:

```
app/
├── routes.js             // route config
├── root.jsx              // app shell
└── routes/
    ├── _index.jsx        // /
    ├── about.jsx         // /about
    └── posts.$id.jsx     // /posts/:id
```

Loaders — fetch on the server, render on arrival

Each route file exports a loader. RR runs it before rendering. Component reads the result with useLoaderData.

```
// app/routes/posts.$id.jsx
export async function loader({ params }) {
  const post = await db.posts.findById(params.id);
  if (!post) throw new Response('Not found', { status: 404 });
  return { post };
}

export default function Post() {
  const { post } = useLoaderData();
  return <Article post={post} />;
}
```

Loaders run on the server in framework mode (or in the browser in library mode if you opt in). Their return value is cached per-route and available via useLoaderData.

Actions — mutate on the server

Forms POST to the route's action. The route revalidates after success.

```
// app/routes/posts.new.jsx
export async function action({ request }) {
  const formData = await request.formData();
  const post = await db.posts.create({
    title: formData.get('title'),
    body:  formData.get('body'),
  });
  return redirect(`/posts/${post.id}`);
}

export default function NewPost() {
  return (
    <Form method='post'> ... </Form>
  );
}
```

Layouts and nested routes

A layout is just a route file with an Outlet. Children render inside it.

```
// app/routes.js
export default [
  layout('routes/dashboard.jsx', [
    index('routes/dashboard.home.jsx'),
    route('reports',
'routes/dashboard.reports.jsx'),
    route('settings',
'routes/dashboard.settings.jsx'),
  ]),
];
```

```
// dashboard.jsx
export default function
DashLayout() {
  return (
    <div>
      <Sidebar />
      <main>
        <Outlet />
      </main>
    </div>
  );
}
```

Protected routes — the loader pattern

Auth check runs in the loader. Redirect from there. The component never renders for unauthenticated users.

```
// app/routes/dashboard.jsx
export async function loader({ request }) {
  const user = await getUser(request);
  if (!user) {
    const url = new URL(request.url);
    throw redirect(`/login?next=${url.pathname}`);
  }
  return { user };
}

export default function Dashboard() {
  const { user } = useLoaderData();
  return <h1>Welcome, {user.name}</h1>;
}
```

Part 2

Next.js App Router

Folder is API. Server Components by default. Server Actions baked in.

Folder is API

Every folder under app/ is a route segment. Special files have special roles.

```
middleware.js                  // project root, NOT under app/
app/
├── layout.jsx                 // shared shell
├── page.jsx                   // /
├── posts/
│   ├── page.jsx               // /posts
│   └── [id]/
│       └── page.jsx           // /posts/:id
└── (auth)/                    // route group, no URL impact
    ├── login/page.jsx
    └── signup/page.jsx
```

The special files

Each file in a route segment plays a specific role.

`page.jsx`

The route's UI. Required if the segment is reachable as a URL.

`layout.jsx`

Wraps the segment and its children. Persists across navigations within the segment.

`loading.jsx`

The Suspense fallback for the segment. Shows while the page streams.

`error.jsx`

Error boundary for the segment. Receives the error and a reset function.

`not-found.jsx`

Renders when notFound() is called or no route matches.

`route.js`

An API endpoint instead of a page. Exports GET, POST, etc.

Server Components by default

page.jsx is a Server Component unless you opt out with 'use client'.

```jsx
// app/posts/[id]/page.jsx
import { notFound } from 'next/navigation';
import { db } from '@/lib/db';

export default async function PostPage({ params }) {
  const post = await db.posts.findById(params.id);
  if (!post) notFound();

  return (
    <article>
      <h1>{post.title}</h1>
      <p>{post.body}</p>
    </article>
  );
}
```

Note: the function is async. No useState, no useEffect — this code runs on the server. Module 6 goes deeper.

Server Actions in routes

Functions marked 'use server' run on the server when invoked from the client.

```
// app/posts/new/page.jsx
import { redirect } from 'next/navigation';

async function createPost(formData) {
  'use server';
  const post = await db.posts.create({
    title: formData.get('title'),
    body:  formData.get('body'),
  });
  redirect(`/posts/${post.id}`);
}

export default function NewPost() {
  return <form action={createPost}> ... </form>;
}
```

Nested layouts in Next.js

layout.jsx in any segment wraps that segment's pages. children prop receives the next layer down.

```
// app/dashboard/layout.jsx
export default function DashboardLayout({ children }) {
  return (
    <div className='dashboard'>
      <Sidebar />
      <main>{children}</main>
    </div>
  );
}

// /dashboard/reports renders DashboardLayout > ReportsPage
// /dashboard/settings renders DashboardLayout > SettingsPage
```

Protected routes — middleware pattern

Next.js middleware runs before any route handler. Redirect there for auth.

```
// middleware.js (project root)
import { NextResponse } from 'next/server';

export function middleware(request) {
  const token = request.cookies.get('session');
  if (!token) {
    const url = request.nextUrl.clone();
    url.pathname = '/login';
    return NextResponse.redirect(url);
  }
}

export const config = { matcher: ['/dashboard/:path*', '/account/:path*'] };
```

Part 3

Side-by-Side

The same three tasks in both frameworks. The differences are smaller than you'd think.

Same task — protected route

Redirect unauthenticated users to /login.

React Router v7

```
// routes/dashboard.jsx
export async function loader({ request
}) {
  const user = await getUser(request);
  if (!user) throw redirect('/login');
  return { user };
}

export default function Dashboard() {
  const { user } = useLoaderData();
  return <h1>Hi, {user.name}</h1>;
}
```

Next.js App Router

```
// middleware.js
export function middleware(request) {
  if (!request.cookies.get('session'))
    return NextResponse.redirect(
      new URL('/login', request.url));
}

// app/dashboard/page.jsx
export default async function
Dashboard() {
  const user = await getUser();
  return <h1>Hi, {user.name}</h1>;
}
```

Same task — load a post

Fetch by id, render, 404 on missing.

React Router v7

```
// routes/posts.$id.jsx
export async function loader({ params })
{
  const post = await db.posts
    .findById(params.id);
  if (!post) throw new Response('', {
status: 404 });
  return { post };
}

export default function Post() {
  const { post } = useLoaderData();
  return <Article post={post} />;
}
```

Next.js App Router

```
// app/posts/[id]/page.jsx
import { notFound } from
'next/navigation';

export default async function Post({
params }) {
  const post = await db.posts
    .findById(params.id);
  if (!post) notFound();

  return <Article post={post} />;
}
```

Same task — create a post

Form submits to the server, redirects on success.

React Router v7

```
// routes/posts.new.jsx
export async function action({ request
}) {
  const fd = await request.formData();
  const post = await db.posts.create({
    title: fd.get('title'),
  });
  return redirect(`/posts/${post.id}`);
}

export default function NewPost() {
  return <Form method='post'>...</Form>;
}
```

Next.js App Router

```
// app/posts/new/page.jsx
async function createPost(fd) {
  'use server';
  const post = await db.posts.create({
    title: fd.get('title'),
  });
  redirect(`/posts/${post.id}`);
}

export default function NewPost() {
  return <form
action={createPost}>...</form>;
}
```

Ergonomics scorecard

Same outcomes, different sensibilities.

Dimension	React Router v7	Next.js App Router
File-based routes	Yes (config or convention)	Yes (folder convention)
Server data loading	loader export	async page component
Server mutations	action export + <Form>	'use server' fn + <form action>
Route protection	redirect from loader	middleware.js + matcher config
Layouts	Outlet + routes config	layout.jsx convention
Server Components	Supported, opt-in	Default, opt-out with 'use client'
Deploy story	Any Node host	Vercel-flavored, runs elsewhere

When to pick which

Decision factors that actually matter in practice.

Reach for RR v7 when…

- You're allergic to Vercel-flavored conventions
- You deploy to AWS / GCP / your own Node hosts and want a clean self-hosted story
- You want minimal magic — explicit loaders and actions feel right
- You're already on React Router v6 and the migration is small
- You want the bridge: library mode for some areas, framework for others

Reach for Next.js when…

- You want the largest ecosystem — most examples, most plugins, most StackOverflow answers
- Edge runtime, ISR, partial prerendering matter
- You're hiring — Next.js is a more common skill on resumes
- You're already on Vercel and want zero-config deploys
- Your app needs heavy SEO and you want streaming + RSC by default

Briefly: TanStack Router

Worth knowing about. Not what we'll teach.

TanStack Router

Type-safe by default

URL params, search params, and loader data are all typed end-to-end. The compile catches typo'd route names.

File-based or code-based — your call

Works as a SPA, an SSR app via TanStack Start, or as a sub-router inside another framework.

Why we don't teach it (yet)

Smaller ecosystem, less Server Components support, your team probably hasn't used it. Worth keeping an eye on.

Coming up: Lab 2

Implement the same auth flow in both frameworks.

Lab 2: Routing + Auth in Two Frameworks

105 minutes · Hands-on · Two reference solutions (RR v7 + Next.js)

What you'll do:

- Implement /login, /signup, /logout, and a protected /home route in React Router v7 framework mode
- Port the same feature to Next.js App Router with middleware-based auth
- Both clients hit the same Express backend you've used in earlier labs
- Write a one-paragraph reflection comparing the two
- Stretch: add a protected nested route with a streaming layout

Module 3 takeaways

What to remember when you're back at your desk.

- React Router v7 is RR + Remix merged. Library mode = old RR. Framework mode = Remix-style.
- Loaders fetch on the server. Actions mutate on the server. Both are per-route exports.
- Next.js page.jsx is a Server Component. The function can be async; useState and useEffect aren't allowed.
- RR protects routes via loader redirects. Next.js protects routes via middleware.
- The shape of both frameworks is similar — the conventions and deploy story differ.

End of Day 1 lecture content

Lab 2 fills the rest of the afternoon. Day 2 starts with state management.

Day 1 covered:

- The 2026 React landscape — what's new in 19, the Compiler, framework picks
- Hooks that matter — the rules, the foundational hooks, the four new ones, specialty hooks
- Routing — RR v7 framework mode and Next.js App Router, side by side
- Lab 1: modernized a class-component chat app and reviewed the AI's work
- Lab 2: implemented the same routing in both frameworks (in progress)

Tomorrow: data and state, server components, the boundary mistakes AI tools make.

Module 4

State Management Without Dogma

Pick the right tool for the job. "Global state for everything" is no longer the default.

60 minutes · Lecture + demos · Lab 3 follows

Too many state-management options

Six things called 'state management', solving five different problems.

useState / useReducer

Local component state. Built in. Often the right answer.

Context API

Pass things through the tree without props. Not a store.

Zustand

The minimal client store. ~1KB. Hooks-shaped API.

Jotai

Atomic state. Fine-grained subscriptions. Recoil's spiritual successor.

Redux Toolkit + RTK Query

The full-featured option. Verbose but complete.

TanStack Query

Server state, masquerading as state management. Often eats half your store.

Part 1

The Decision Framework

Where does this state live? Most of the rest follows from there.

Start with one question

Where does this state actually live?

Where does this state live?

If it lives on the server,

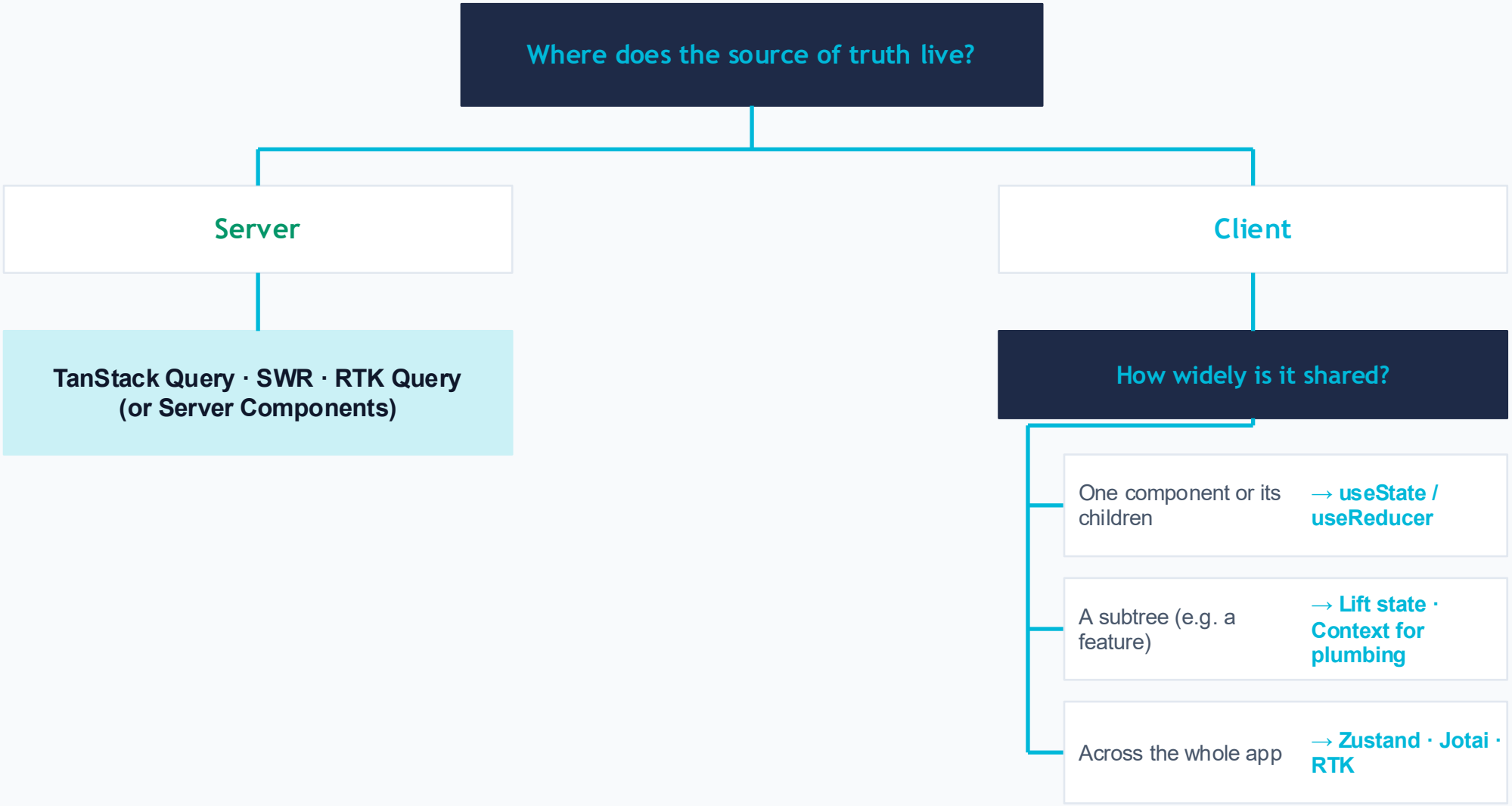
…you have server state. Use TanStack Query, SWR, RTK Query, or Server Components. Don't put server data in a client store unless you've thought hard about it.

If it lives in the client,

…you have client state. The next question becomes: how many components share it?

The decision tree

Five questions, six answers. Most state lands in the first two boxes.



Part 2

Foundational Patterns

useState, useReducer, Context — and the antipatterns that keep them honest.

useState and useReducer — the right defaults

Most state in a React app is local. Resist the upgrade reflex.

Reach for useState first

If a component reads or writes the value, that's where the state belongs. Don't lift until you have to.

Use functional updaters

setX(n => n + 1) when the next value depends on the previous. Race-safe under concurrent updates.

Lazy init for expensive defaults

useState(() => expensive()) runs the function once on mount, not on every render.

useReducer when transitions get complex

Multiple related fields, validation, undo/redo, state machines. The transition function tests in isolation.

Context — plumbing, not a store

Pass values through the tree without props. That's it. Everything else is misuse.

Good fits

- Theme (dark/light, brand colors)
- Authenticated user object
- i18n locale + translation function
- Feature flags
- A pre-instantiated client (e.g., apollo, query client)

Bad fits

- Frequently-changing values that re-render the whole tree
- A list of items most of the tree doesn't read
- Server data — use TanStack Query instead
- Anything you'd need to subscribe to a slice of (use a store)
- "State management" — context is plumbing, not management

The context-as-store antipattern

Every consumer re-renders when ANY value changes. At scale, this is a perf bug waiting.

The trap

```
// One context for everything
const AppCtx = createContext(null);

function AppProvider({ children }) {
  const [user, setUser] = useState();
  const [theme, setTheme] = useState();
  const [posts, setPosts] = useState([]);
  const [drawer, setDrawer] = useState();
  return <AppCtx.Provider value={{
    user, setUser, theme, setTheme,
    posts, setPosts, drawer, setDrawer
  }}>{children}</AppCtx.Provider>;
}
```

Why it bites

- Every setState rebuilds the value object
- Every consumer re-renders, even if it only read user.name
- Memoization helps but the object changes anyway
- Adding a new field means re-rendering the whole tree

If you find yourself adding a fifth value to one context, it's a store now. Reach for one.

Part 3

Client Stores

Zustand, Jotai, Redux Toolkit. Same problem space, three sensibilities.

When to reach for a store

Not every app needs one. Most that do, need a small one.

- Multiple distant components read or write the same value
- You want subscribers to render only when their slice changes
- Actions need to be testable in isolation, separately from the UI
- You want time-travel debugging or middleware (logging, persistence)
- Your context provider has grown more than four or five values

None of the above? Stick with useState. The simplest tool that works is the right one.

Zustand — the minimal store

~1KB. A hook factory. Subscribers re-render only when their slice changes.

```
import { create } from 'zustand';

const useCart = create((set) => ({
  items: [],
  add: (item) => set((s) => ({ items: [...s.items, item] })),
  remove: (id) => set((s) => ({ items: s.items.filter(i => i.id !== id) })),
  clear: () => set({ items: [] }),
}));

// in any component
function CartCount() {
  const count = useCart((s) => s.items.length);
  return <span>{count}</span>;
}
```

Zustand — what you get and don't

The minimum-surface-area store. Sometimes you want more.

Strengths

- Tiny — bundle and mental footprint both
- Selectors give per-slice subscriptions for free
- No Provider needed in your tree
- Easy to test the store in isolation
- Plays well with TS and the React Compiler

Trade-offs

- No formal action type — actions are just methods
- Less convention for a large team — easy to drift
- No built-in middleware ecosystem to match Redux
- DevTools work, but the trace isn't as rich as Redux's

Jotai — atomic state

State as small atoms you compose.

```
import { atom, useAtom } from 'jotai';

// primitive atoms
const cartItemsAtom = atom([]);
const promoCodeAtom = atom('');

// derived atom (read-only)
const cartTotalAtom = atom((get) => {
  const items = get(cartItemsAtom);
  const promo = get(promoCodeAtom);
  return calcTotal(items, promo);
});

// in a component
const total = useAtomValue(cartTotalAtom);
```

Jotai — what you get and don't

If your state is naturally graph-shaped, this fits. If it's flat, this is overkill.

Strengths

- Fine-grained subscriptions per atom
- Derived atoms model dependencies cleanly
- Async atoms integrate with Suspense naturally
- Excellent TS inference — atom types compose

Trade-offs

- Mental model differs from Redux/Zustand — onboarding cost
- DevTools are functional but less mature
- Big atom graphs can become hard to reason about
- Smaller community — fewer middleware options

Redux Toolkit + RTK Query

The full-featured option. Verbose, opinionated, complete.

```
// store/cartSlice.js
import { createSlice } from '@reduxjs/toolkit';

const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [] },
  reducers: {
    add: (state, action) => { state.items.push(action.payload); },
    remove: (state, action) => {
      state.items = state.items.filter(i => i.id !== action.payload);
    },
  },
});

export const { add, remove } = cartSlice.actions;
export default cartSlice.reducer;
```

RTK Query — the bundled server-state piece

If you're already on RTK, you may not need TanStack Query.

```
import { createApi, fetchBaseQuery } from '@reduxjs/toolkit/query/react';

export const api = createApi({
  baseQuery: fetchBaseQuery({ baseUrl: '/api' }),
  tagTypes: ['Posts'],
  endpoints: (build) => ({
    listPosts: build.query({ query: () => '/posts', providesTags: ['Posts'] }),
    createPost: build.mutation({
      query: (body) => ({ url: '/posts', method: 'POST', body }),
      invalidatesTags: ['Posts'],
    }),
  }),
});
export const { useListPostsQuery, useCreatePostMutation } = api;
```

Three stores at a glance

Pick by team size, app shape, and the value of conventions.

Dimension	Zustand	Jotai	Redux Toolkit
Bundle size	~1KB	~3KB	~12KB + middleware
API surface	Small	Small	Medium-large
Conventions	Loose	Atom-based	Strong (slices, selectors, tags)
DevTools	Good	Functional	Best in class
Server state	BYO (e.g. TanStack Query)	BYO or async atoms	Bundled (RTK Query)
Best for	Most apps	Graph-shaped state	Large teams, conventions matter

Part 4

Server State

If the data lives on the server, treat it differently. Half your old store probably belongs here.

Server state is different

Not your state. Not your source of truth. A cached view of someone else's truth.

It can change behind your back
Another user, another tab, a background job — the truth lives elsewhere.

It needs invalidation, not just updates
Your local edit is provisional until the server confirms.

Loading and error are first-class concerns
The wire is slow and unreliable. Don't pretend otherwise.

It deduplicates across the app
Two components asking for the same posts shouldn't fire two requests.

TanStack Query — the canonical lever

Module 5 covers it in depth. Here, just see the shape.

```
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';

function PostList() {
  const { data, isPending, error } = useQuery({
    queryKey: ['posts'],
    queryFn: () => api.listPosts(),
  });

  if (isPending) return <Skeleton />;
  if (error) return <Error error={error} />;
  return <ul>{data.map(p => <Post key={p.id} {...p} />)}</ul>;
}
```

Why this changes everything

When you split server state out, your client store gets dramatically smaller.

Before — everything in the store

- Auth user object (server)
- Posts list (server)
- Single post by id (server)
- Drafts being edited (client)
- Sidebar open/closed (client)
- Theme (client)
- Notifications panel (server)
- Filter selections (client)

After — split by where truth lives

TanStack Query (server)

User · Posts · Single post · Notifications

Client store (or just useState)

Drafts · Sidebar · Theme · Filters

Part 5

Picking and Defending

The framework on one slide. Then the lab.

The decision framework — one more time

Five questions you ask, in order, for any piece of state.

1	**Where does the source of truth live?**	*Server → TanStack Query / RTK Query / Server Component.*
2	**Used in only one component or its children?**	*Yes → useState or useReducer.*
3	**Could you lift it to a common ancestor?**	*Yes, and it stays small → just lift. Add Context for plumbing.*
4	**Is it shared widely across the app?**	*Yes → reach for a store.*
5	**Which store?**	*Most: Zustand. Graph-shaped: Jotai. Large team + conventions: RTK.*

AI mistakes in state management

What to watch for when you ask AI to add state to your app.

- Reaching for Redux when useState would do — over-engineering
- Putting server data in a client store — duplicates, stale-state bugs
- Context with five values + setters — re-render storm
- useState mirroring props that should just be calculated
- Multiple stores when one slice would do
- localStorage for everything, including sensitive tokens

Module 4 takeaways

What to remember when you're back at your desk.

- Start with the question 'where does this state live?' — server vs client decides half the choice.
- useState is still the right answer for most state. Don't upgrade reflexively.
- Context is plumbing, not a store. If you have five values in a context, that's a store now.
- Zustand is the right default client store for most apps.
- TanStack Query (or RTK Query) eats half of what people used to put in Redux.

Coming up: Lab 3

Pick a tool. Refactor the social-media app's state. Defend your choice in 60 seconds.

Lab 3: Pick-Your-Tool State Refactor

90 minutes · Hands-on · Three reference solutions (one per tool)

What you'll do:

- Start with the social-media app — messy prop-drilling and ad-hoc context overuse
- Pick one of: Zustand, Jotai, or RTK + RTK Query
- Refactor the state with your chosen tool
- Present a 60-second pitch on why your tool was right for this app
- Stretch: do it again with a second tool and write a comparison

After the lab: Module 5

Modern data fetching — TanStack Query in depth.

Module 5 picks up where the server-state section ended:

- The four flavors of data fetching today— client fetch, TanStack Query, Suspense + use(), server-side
- TanStack Query in depth — queries, mutations, invalidation, optimistic updates
- Error boundaries and retry strategies that don't lie to your users
- Lab 4: convert the social-media app's ad-hoc fetch calls to TanStack Query, with optimistic updates

Module 5

Modern Data Fetching

Four flavors. One canonical client tool. A clear story for loading and errors.

60 minutes · Lecture + demos · Lab 4 follows

Four flavors of data fetching

Each one solves a real problem. The skill is matching them to the situation.

Client fetch + useEffect

The from-scratch baseline. Works, but you write everything: loading, error, retry, cache.

TanStack Query / SWR / RTK Query

The canonical client-driven option. Cache, dedupe, retry, invalidation — solved.

Suspense + use()

The React-native flavor. Promise in, value out. Pairs with route-level data fetching.

Server-side (RSC + loaders)

Fetch on the server, render on arrival. No client-side fetch at all for this data.

Part 1

The Four Flavors

What each one is for. Where each one fails. How to pick.

Client fetch + useEffect — the baseline

Works for one component. Falls apart at scale.

```
function Posts() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch('/api/posts')
      .then((r) => r.json())
      .then(setPosts)
      .catch(setError)
      .finally(() => setLoading(false));
  }, []);
  // … render
}
```

What this doesn't do:

- No cache — visiting twice fetches twice
- No deduplication — two components fetch the same URL twice
- No retry on transient failures
- No invalidation after a mutation
- No background refresh — stale forever
- Race conditions if effect re-runs while old request pending

TanStack Query — the canonical client option

Same problem, three lines instead of fifteen. Plus everything client fetch didn't do.

```
function Posts() {
  const { data, isPending, error } = useQuery({
    queryKey: ['posts'],
    queryFn: () => fetch('/api/posts').then((r) => r.json()),
  });

  if (isPending) return <Skeleton />;
  if (error) return <Error error={error} />;
  return <PostList posts={data} />;
}
```

Free with this: cache, dedup, retry, invalidation, background refetch, stale-while-revalidate, focus refetch, network reconnect refetch.

Suspense + use() — the React-native flavor

Pass a promise in, get a value out. The component suspends while resolving.

```
// page (Server Component or parent that creates the promise once)
function PostsPage() {
  const postsPromise = fetchPosts();
  return (
    <Suspense fallback={<Skeleton />}>
      <Posts postsPromise={postsPromise} />
    </Suspense>
  );
}

// child
function Posts({ postsPromise }) {
  const posts = use(postsPromise);
  return <PostList posts={posts} />;
}
```

Critical: don't create the promise inside the component that calls use(). New promise every render → infinite Suspense loop.

Server-side — fetch where the data lives

Server Components and route loaders run before the page reaches the client.

Next.js Server Component

```
// app/posts/page.jsx
export default async function Posts() {
  const posts = await db.posts.list();
  return <PostList posts={posts} />;
}
```

RR v7 framework loader

```
// app/routes/posts.jsx
export async function loader() {
  return { posts: await db.posts.list()
};
}
export default function Posts() {
  const { posts } = useLoaderData();
  return <PostList posts={posts} />;
}
```

Picking a flavor

Default to TanStack Query. Use the others when they earn it.

Situation	Pick
Default for client-driven data	`TanStack Query`
Data lives on the server, page-shaped	`Server Components`
Per-route, framework-managed data	`Route loader (RR v7)`
Promise-shaped data already in hand	`use() with Suspense`
Embedded widget, can't add a library	`Plain client fetch (carefully)`
Already on Redux Toolkit	`RTK Query`

Part 2

TanStack Query in Depth

Setup, queries, mutations, invalidation, optimistic updates.

Setup — one QueryClient per app

Wrap your tree once. The provider gives every descendant access to the cache.

```
// app/main.jsx
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 30 * 1000,        // 30s — adjust per app
      retry: 2,
      refetchOnWindowFocus: true,
    },
  },
});

createRoot(document.getElementById('root')).render(
  <QueryClientProvider client={queryClient}>
    <App />
  </QueryClientProvider>
);
```

useQuery — read data

queryKey identifies the data. queryFn fetches it. Everything else has good defaults.

```
function PostDetail({ id }) {
  const { data, isPending, isError, error, refetch } = useQuery({
    queryKey: ['posts', id],          // unique cache key
    queryFn: () => api.getPost(id), // must return a promise
    staleTime: 60 * 1000,           // override per-query
    enabled: !!id,                  // skip if no id yet
  });

  if (isPending) return <Skeleton />;
  if (isError) return <ErrorBox error={error} onRetry={refetch} />;
  return <Article post={data} />;
}
```

Query keys are the cache contract

Two queries with the same key share a cache entry. Get the keys right and everything else falls into place.

Conventions worth adopting

`Hierarchical: ['posts'], ['posts', id], ['posts', id, 'comments']`

Lets you invalidate ['posts'] to wipe everything posts-related.

`Include parameters: ['posts', { search, page }]`

Different searches get different cache entries. Identical params share.

`Avoid mutable references: ['posts', new Date()]`

New key every render — cache miss every time.

`Centralize key factories: queryKeys.posts.detail(id)`

Refactor-friendly. Catches typos at build time.

useMutation — write data

mutationFn does the writing. onSuccess invalidates queries. The hook tracks pending and error.

```
function NewPostForm() {
  const queryClient = useQueryClient();

  const { mutate, isPending, isError, error } = useMutation({
    mutationFn: (newPost) => api.createPost(newPost),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['posts'] });
    },
  });

  function handleSubmit(formData) {
    mutate({ title: formData.get('title'), body: formData.get('body') });
  }

  return <form action={handleSubmit}> ... <button
disabled={isPending}>Post</button></form>;
}
```

Invalidation — the heart of cache management

After a mutation, tell the cache what's stale. TanStack refetches in the background.

Invalidate one entry

```
queryClient.invalidateQueries({ queryKey: ['posts', id] });
```

Invalidate a hierarchy

```
queryClient.invalidateQueries({ queryKey: ['posts'] });
```

Invalidate by predicate

```
queryClient.invalidateQueries({ predicate: (q) => q.state.data?.authorId === userId });
```

Refetch immediately (no stale wait)

```
queryClient.refetchQueries({ queryKey: ['posts'] });
```

Optimistic updates — the pattern

Show the success state immediately. Roll back on error. Reconcile on settle.

The four-step pattern in onMutate / onError / onSettled:

1. onMutate: cancel in-flight queries for this key, snapshot the cache, write the optimistic value.
2. Return the snapshot from onMutate so onError can use it for rollback.
3. onError: restore the snapshot. The user sees the rollback, not a stale optimistic value.
4. onSettled: invalidate the key so the cache catches up to the server's truth.

Optimistic updates — the code

The classic 'add post' flow with rollback on error.

```
useMutation({
  mutationFn: api.createPost,
  onMutate: async (newPost) => {
    await queryClient.cancelQueries({ queryKey: ['posts'] });
    const snapshot = queryClient.getQueryData(['posts']);
    queryClient.setQueryData(['posts'], (old = []) => [{ ...newPost, sending: true
}, ...old]);
    return { snapshot };
  },
  onError: (err, newPost, ctx) => {
    queryClient.setQueryData(['posts'], ctx.snapshot);
    toast.error(err.message);
  },
  onSettled: () => {
    queryClient.invalidateQueries({ queryKey: ['posts'] });
  },
});
```

Part 3

Loading and Errors

Two patterns. Pick the one that fits the surrounding code.

Loading — flags or Suspense

Both work with TanStack Query. Pick by what surrounds your component.

Status flags

```
const { data, isPending } =
  useQuery(...);
if (isPending) return <Spinner />;
```

- Component owns its loading UI
- Easy to add per-query custom states
- Default — pick this unless you have a reason

Suspense mode

```
const { data } = useSuspenseQuery(...);
// no isPending check — parent
// Suspense boundary handles it
```

- Loading UI lives in a parent <Suspense>
- Component code reads as if data is always present
- Fits naturally with route-level loading.jsx files

Errors — boundaries beat conditionals

Wrap features in error boundaries. TanStack Query throws into them when configured.

```
// app/main.jsx
<QueryErrorResetBoundary>
  {({ reset }) => (
    <ErrorBoundary onReset={reset} fallbackRender={({ error, resetErrorBoundary }) => (
      <ErrorPanel error={error} onRetry={resetErrorBoundary} />
    )}>
      <App />
    </ErrorBoundary>
  )}
</QueryErrorResetBoundary>

// in the query
useQuery({ ..., throwOnError: true });
```

Retry — when, when not

Default retries 3 times with exponential backoff. Override per-query.

Retry transient network errors
5xx, fetch failures, timeouts. Default behavior is fine.

Don't retry 4xx errors
Client mistakes won't fix themselves on retry. retry: (n, err) => err.status < 500.

Don't retry mutations by default
A failed POST shouldn't auto-fire again — could double-submit. retry: false on mutations.

Custom backoff for rate-limited APIs
retryDelay: (attempt) => Math.min(1000 * 2 ** attempt, 30000).

Part 4

use() vs useQuery

Different tools, overlapping problems. Knowing when each fits is the skill.

use() and TanStack Query — when each wins

Both can read promises. Pick by what owns the promise and what you need around it.

`use(promise)`

When

Promise is created by a route loader, Server Component, or stable parent.

Win

Components read like sync code. No isPending checks. Composes naturally with Suspense.

Don't get

Cache, dedup, retry, invalidation, refetch. You'd build those yourself.

`useQuery / useMutation`

When

Component triggers the fetch. Multiple components share the data. Mutations and refetches matter.

Win

Cache, dedup, retry, invalidation, refetch. The full data layer.

Don't get

The 'reads like sync code' ergonomic of use() — but useSuspenseQuery gets close.

Module 5 takeaways

What to remember when you're back at your desk.

- Default to TanStack Query for client-driven data. It solves the dozen problems plain fetch + useEffect doesn't.
- Query keys are the cache contract. Hierarchical, parameterized, never mutable. Centralize them.
- Mutations finish with invalidation. The whole pattern is: do the thing, then tell the cache.
- Optimistic updates have four phases: cancel → snapshot → write → reconcile. Don't memorize, recognize.
- Status flags or Suspense — pick by what surrounds the component. Errors belong in boundaries.
- use() wins when a parent owns the promise. Server Components win when the data lives on the server.

AI mistakes in data fetching

What to look for when AI helps you wire up data.

- Reaching for useEffect + fetch instead of useQuery — old pattern from training data
- Recreating the QueryClient on every render — multiple caches, broken dedup
- Mutable query keys: ['posts', new Date()] — cache miss every render
- Missing invalidation after a mutation — stale UI until next manual refetch
- Retrying mutations by default — risk of double-submits
- Promise created inside the use() consumer — infinite Suspense loop

Coming up: Lab 4

Convert the social-media app's ad-hoc fetch calls to TanStack Query.

Lab 4: TanStack Query Migration

75 minutes · Hands-on · Solution: solutions/lab-04-tanstack-query/

What you'll do:

- Replace ad-hoc fetch + useEffect calls in the social-media client with useQuery
- Convert the create-post flow to a useMutation with invalidation
- Add an optimistic update — posts appear instantly
- Wrap the feed in <Suspense> + an ErrorBoundary for proper loading/error UX
- Stretch: implement infinite scroll for the posts feed using useInfiniteQuery

After the lab: Module 6

Server Components and Server Actions in depth.

Module 6 picks up the server-side flavor we touched on today:

- Server Components — what runs where, what props can cross the boundary
- Server Actions with progressive enhancement and useActionState
- Streaming with multiple Suspense boundaries
- The boundary mistakes AI tools love to make — and how to spot them
- Lab 5: build a Server Components dashboard with a Server Action form

End of Module 5

Day 2 still has Module 6 + Lab 5 after Lab 4.

Where we are in Day 2:

- Module 4 — State management decision framework ✓
- Lab 3 — Pick-your-tool state refactor ✓
- Module 5 — Modern data fetching ✓ (just finished)
- Lab 4 — TanStack Query migration (next)
- Module 6 — Server Components + Server Actions
- Lab 5 — Server Components dashboard

Module 6

Server Components & Server Actions

Where code runs is the new mental model. Get the boundary right and the rest follows.

75 minutes · Lecture + demos · Lab 5 follows

The mental model

Every component runs somewhere. The job is knowing where, and what's allowed there.

Server

- Runs in your Node process (or edge runtime)
- Has access to the database, file system, secrets
- Renders to a streamed payload
- No browser, no DOM, no event handlers

Client (Browser)

- Runs in the user's browser tab
- Has DOM, event handlers, browser APIs
- Hydrates from the server's payload
- Can't access your DB or your secrets

Part 1

Server Components

Components that only run on the server. The default in modern frameworks.

What's a Server Component?

A component that runs on the server, returns serializable output, and ships zero JS to the browser.

```jsx
// app/posts/page.jsx — Next.js Server Component
import { db } from '@/lib/db';

export default async function PostsPage() {
  const posts = await db.posts.findMany();
  return (
    <main>
      {posts.map((p) => <article key={p.id}>{p.title}</article>)}
    </main>
  );
}
```

The function is async. Database access at the top of the component. Zero JS in the browser for this code.

Server first is the new default

In Next.js 15 and RR v7 framework mode, every component is a Server Component unless you opt out.

The shift in defaults

All components are server components by default

page.jsx, layout.jsx, and any component they import — server unless marked otherwise.

Opt in to client with 'use client'

First line of a file. Everything in that file (and its imports) is now client-side.

Client components import client components

Once you cross the boundary into client code, the rest of that subtree is client too.

The boundary is per-file, not per-component

You can't have a server function and a client function in the same file.

What Server Components can do

Things you couldn't do in a Client Component without a separate API layer.

- Query the database directly — no API endpoint needed
- Read files from disk, including markdown and config
- Use environment variables and secrets without exposing them
- Call private internal services that aren't internet-accessible
- Run heavy computations without shipping the JS to the browser
- Import server-only libraries (e.g., Node crypto, fs, ORM clients)

What Server Components can't do

Anything that needs the browser. The list is shorter than you'd think.

- Use useState, useEffect, useReducer, or any hook with state
- Attach event handlers (onClick, onChange, etc.)
- Use browser APIs — window, document, localStorage, fetch
- Use refs — useRef won't work without state to back it
- Be marked 'use client' — that flips the file to client
- Be re-rendered on the client — they render once, on the server

The 'use client' directive

First line of a file. Everything in that file is client-side from there.

```
'use client';

// Now we can use hooks, event handlers, browser APIs
import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(n => n + 1)}>
      Clicked {count} times
    </button>
  );
}
```

The directive is per-FILE. Once present, every export from that file is a Client Component, and every component imported into the file becomes client.

The client/server boundary

Components 'cross' from server into client. They never cross back.

Server Components

```
<Page>
  ↓
<Layout>
  ↓
<PostsList>  ← awaits db
  ↓
<Post> <Post> <Post>
```

crosses

Client Components

```
<LikeButton>
  ↓
useState, onClick

<CommentBox>
  ↓
useState, useFormStatus
```

Server components can render client components as children. Client components cannot import server components.

What can cross the boundary

Props from a Server Component to a Client Component must be serializable.

Allowed props

- Strings, numbers, booleans, null, undefined
- Plain objects and arrays of the above
- Date, Map, Set, Promise (yes — promises serialize)
- Server Action functions (passed by reference, executed on server)
- JSX as a children prop (renders separately)

Not allowed

- Regular functions defined inline (not Server Actions)
- Class instances with methods
- Symbols (other than well-known)
- Functions returned from libraries (e.g., a configured logger)
- DOM elements or refs

Composition: pass Client Components as children

A Client Component can render Server Component children — they just need to be passed in, not imported.

```
// AccordionPanel.jsx — Client Component
'use client';
export default function AccordionPanel({ children, summary }) {
  const [open, setOpen] = useState(false);
  return <details onClick={() => setOpen(o => !o)}><summary>{summary}</summary>{open && children}</details>;
}

// page.jsx — Server Component using it
import AccordionPanel from './AccordionPanel';
import PostsList from './PostsList';   // also a Server Component

export default async function Page() {
  return (
    <AccordionPanel summary='Recent posts'>
      <PostsList />     // Server Component as a child of a Client Component
    </AccordionPanel>
  );
}
```

Part 2

Server Actions

Functions that run on the server when called from the client. Forms become quiet again.

What's a Server Action?

A function marked 'use server'. Called from the client; runs on the server.

```
// app/posts/new/page.jsx
import { redirect } from 'next/navigation';
import { db } from '@/lib/db';

async function createPost(formData) {
  'use server';
  const post = await db.posts.create({
    data: { title: formData.get('title'), body: formData.get('body') },
  });
  redirect(`/posts/${post.id}`);
}

export default function NewPost() {
  return <form action={createPost}> ... </form>;
}
```

Server Actions in forms

Pass the action to <form action>. React handles the rest.

FormData is the input

Action receives a FormData object. No JSON parsing, no useState for fields.

Submission is intercepted

React posts to the action; no full page reload. The page revalidates after.

Pending state is automatic

useFormStatus inside any descendant of the form sees pending: true while the action runs.

Works without JS

If JS hasn't loaded yet, the browser does a regular form POST. Progressive enhancement built-in.

Pair with useActionState

useActionState wraps the action and gives you state, the wrapped action, and isPending.

```
// LoginForm.jsx — Client Component
'use client';
import { useActionState } from 'react';
import { login } from '@/actions/auth';

export default function LoginForm() {
  const [state, action, isPending] = useActionState(login, {});

  return (
    <form action={action}>
      <input name='email' defaultValue={state.email} />
      <input name='password' type='password' />
      {state.error && <p className='error'>{state.error}</p>}
      <button disabled={isPending}>Sign in</button>
    </form>
  );
}
```

Server Actions from event handlers

Not all actions are forms. You can call them from any Client Component handler.

```
// LikeButton.jsx — Client Component
'use client';
import { useTransition } from 'react';
import { likePost } from '@/actions/posts';   // 'use server' fn

export default function LikeButton({ postId, initialLiked }) {
  const [isPending, startTransition] = useTransition();

  function handleClick() {
    startTransition(async () => {
      await likePost(postId);
    });
  }

  return <button onClick={handleClick} disabled={isPending}>♥</button>;
}
```

Part 3

Streaming on the Server

Suspense boundaries split a page into chunks that stream in independently.

The streaming model

The shell ships first. Slow data arrives in its own time. The user sees content sooner.

How it flows

1. **Page request hits the server.**
   The framework starts rendering.

2. **Fast parts render immediately.**
   Header, nav, layout chrome — anything that doesn't await something slow.

3. **<Suspense> boundaries hold a placeholder.**
   Each boundary's fallback ships in the initial HTML. The user sees the page load.

4. **Slow data resolves, the chunk streams in.**
   React replaces the placeholder with the real content as it arrives.

loading.jsx and granular boundaries

Next.js gives each segment a loading.jsx. RR v7 uses defer + Await.

```jsx
// app/dashboard/page.jsx
import { Suspense } from 'react';

export default function Dashboard() {
  return (
    <main>
      <Header />                  // fast

      <Suspense fallback={<RevenueSkeleton />}>
        <Revenue />               // slow query
      </Suspense>

      <Suspense fallback={<ChartSkeleton />}>
        <UserChart />             // even slower
      </Suspense>
    </main>
  );
}
```

Two boundaries, two streams

- Header renders immediately
- Both skeletons appear next
- Revenue arrives, replaces its skeleton
- UserChart arrives later, replaces its skeleton

User sees content faster than a single big-query page would deliver.

Part 4

Boundary Pitfalls

Where teams get this wrong. AI tools especially.

Pitfall: 'use client' too high in the tree

Marking a layout 'use client' opts out the entire subtree. You lose the Server Component benefits.

The mistake

```
// app/dashboard/layout.jsx
'use client';  // ← whole subtree
import { useState } from 'react';

export default function Layout({ children }) {
  const [collapsed, setCollapsed] = useState(false);
  return (
    <div className={collapsed ? 'compact' : ''}>
      <Sidebar />
      <main>{children}</main>
    </div>
  );
}
```

The fix

- Keep the layout server-only
- Extract the interactive part (the collapse toggle) into a small Client Component
- Pass children through the Client Component back into the server tree

Result: the subtree stays Server Components. Only the toggle ships JS.

Pitfall: importing server-only code into a client file

The build error is opaque. The cause is usually a transitive import.

```
// utils.js — has no 'use client'
import { db } from '@/lib/db';   // server-only

export function formatTitle(post) { return post.title.toUpperCase(); }   // pure

// LikeButton.jsx — 'use client'
'use client';
import { formatTitle } from './utils';   // pulls db transitively!
```

Mark server-only files with the 'server-only' package import. Build will refuse to bundle them into client code with a clear error.

Pitfall: non-serializable props across the boundary

Trying to pass a function from a Server Component to a Client Component fails — unless it's a Server Action.

Won't work

```
// page.jsx (server)
function handleSomething() {
  console.log('hi');
}

<ClientButton onClick={handleSomething}
/>
// → Error: cannot pass functions
```

Will work

```
// page.jsx (server)
async function handleSomething() {
  'use server';
  await db.something();
}

<ClientButton
onClick={handleSomething} />
// → Server Action; works
```

Pitfall: hydration mismatches

Server renders one thing, client expects another. Often non-deterministic.

Date.now() in render
Server time != client time. Use a stable key, or render conditionally on client only.

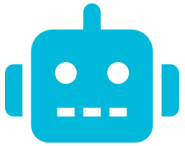
Math.random() in render
Different on each render. Move to useState with initializer or generate on the server.

navigator.* / window.* checks
Undefined on server. Wrap in useEffect or 'use client' boundary.

Locale-dependent formatting
Server locale != browser locale. Use a stable formatter or render on client.

The AI escape-hatch pattern

When AI hits a Server/Client boundary, it slaps 'use client' on top until the error goes away.



What you'll see in AI-generated code

1. AI writes a Server Component that uses useState. Build fails.
2. AI adds 'use client' to the top of the file. Build passes.
3. The component is now a Client Component — and so is everything it imports.
4. The benefits of Server Components are gone. JS bundle grew. Server-only imports break.

The right fix is almost always to extract just the interactive part as a small Client Component and leave the parent server-side.

Part 5

Wrap-up

Two frameworks, same primitives. Then the lab.

Next.js vs RR v7 — same primitives, different syntax

Once you understand the boundary, both frameworks are mostly bookkeeping.

Primitive	Next.js 15	React Router v7
Server Component default	page.jsx, layout.jsx	Routes are server by default
Client opt-in	'use client'	'use client'
Server Action declaration	'use server' inline or in module	'use server' inline or in module
Form action	<form action={fn}>	<form action={fn}> or <Form action>
Streaming primitive	<Suspense> + loading.jsx	<Suspense> + defer/Await
Server Actions on event handlers	Yes — use directly	Yes — call in transition

AI mistakes in Server Components

Your review checklist for Lab 5 and beyond.

- 'use client' added at the file top to silence a hook-related error
- Server-only import (db client, env vars) leaked into a 'use client' file
- Inline event handler passed across the boundary as a regular function
- Date.now() or Math.random() in render — hydration mismatch coming
- The whole page wrapped in one Suspense — no streaming benefit
- Server Action without 'use server' directive — runs as a regular client function

Module 6 takeaways

What to remember when you're back at your desk.

- Server-first is the new default. Add 'use client' only where you need state or events.
- The boundary is per-file. Once a file is 'use client', everything in it (and downstream) is client.
- Server Components can render Client Components. Client Components can receive Server Components as children — never import them.
- Props across the boundary must be serializable. Inline event handlers are the most common violation.
- Server Actions = 'use server' functions. Pair with useActionState for forms, useTransition for handlers.
- Streaming wins come from MULTIPLE Suspense boundaries. One boundary around everything is no win at all.

Coming up: Lab 5

Build a Server Components dashboard with a Server Action form.

Lab 5: Server Components Dashboard

75 minutes · Hands-on · Solution: solutions/lab-05-dashboard/

What you'll do:

- Build a small dashboard page with Server Components fetching data
- Add a Server Action form using useActionState for state and validation
- Identify which components must be 'use client' and document why
- Stretch: add streaming with multiple Suspense boundaries

End of Day 2

Lab 5 closes the day. Day 3 covers performance, testing, and AI.

Day 2 covered:

- State management decision framework + pick-your-tool refactor
- Modern data fetching + TanStack Query migration
- Server Components + Server Actions (just finished)
- Lab 5 — Server Components dashboard (next)

Tomorrow: performance in the Compiler era, testing with Vitest + RTL, and a full module on working with AI assistants.

Module 7

Performance in the Compiler Era

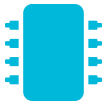
Measure first. Use the Compiler. Reach for hand work only when the data says so.

60 minutes · Lecture + demos · Lab 6 follows

What we'll cover

Five tools, in the order you'd reach for them.

Measurement
Profiler, Lighthouse, Core Web Vitals — before any optimization.



The React Compiler
Auto-memoization. What it does, what it doesn't.

Manual memoization
When useMemo, useCallback, React.memo still earn their keep.

Code splitting
React.lazy + Suspense. Route-level and component-level.

Beyond the render
Virtualization, Web Workers, throttling/debouncing.

Part 1

Measure First

Optimization without measurement is theater. Pick a tool, get a baseline.

The 'no measurement' trap

Most performance work in React 19 is on the wrong thing.

Symptom: 'Add useMemo everywhere, just in case'
Memoization without measurement adds bugs and code, not speed. Most components don't need it.

Symptom: 'It feels slow'
Without numbers, you can't tell if your fix worked. You're chasing a vibe.

Symptom: 'The first thing I tried was code splitting'
If your bottleneck is a render storm, code splitting won't help. Profile first.

Symptom: 'I'll just optimize all the things'
Most components are not the bottleneck. Optimizing 80% of components for 0% gain is what burns weeks.

React DevTools profiler

Records render counts and durations. Tells you which components are actually expensive.

How to use it

- Open the Profiler tab in React DevTools
- Click record, interact with your app, click stop
- The flame graph shows render time per component
- Sort by 'rendered most' to find re-render storms
- Click any bar to see why that component rendered

What to look for

- Components rendering on every keystroke
- A component rendering >10× per user action
- Long render durations (>16ms = dropped frame)
- 'Why did this render?' showing prop changes you didn't intend
- Unexpected re-renders after the Compiler memoizes

Lighthouse + Core Web Vitals

Lighthouse measures the user's experience. Profiler measures your code.

LCP — Largest Contentful Paint

`good: ≤ 2.5s`

How quickly the main content appears. Server Components and streaming help most.

INP — Interaction to Next Paint

`good: ≤ 200ms`

How responsive your app feels. Replaced FID in 2024. Long tasks are the killer.

CLS — Cumulative Layout Shift

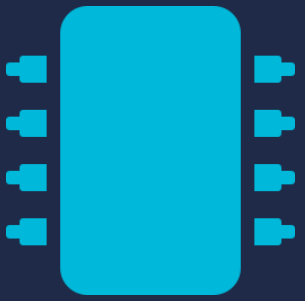
`good: ≤ 0.1`

How stable the layout is. Skeletons that match real content shape avoid this.

TTFB — Time to First Byte

`good: ≤ 800ms`

Server response time. Mostly your backend, but streaming SSR helps.



Part 2

The React Compiler

Auto-memoization. The biggest 2024-to-2026 shift in how we write React.

What the Compiler does

Reads your component, figures out what's safe to memoize, generates the equivalent code.

Before — manual

```
function List({ items, query }) {
  const filtered = useMemo(
    () => items.filter(i =>
i.includes(query)),
    [items, query]
  );
  const onClick = useCallback(
    (e) => save(e), [save]
  );
  return <Items items={filtered}
onClick={onClick} />;
}
export default memo(List);
```

After — Compiler enabled

```
function List({ items, query }) {
  const filtered = items.filter(
    i => i.includes(query)
  );
  const onClick = (e) => save(e);
  return <Items items={filtered}
onClick={onClick} />;
}

// Compiler memoizes equivalently
export default List;
```

What the Compiler doesn't do

Knowing the limits is what keeps performance work honest.

It doesn't fix Rules of Hooks violations	Same rules as before. Components that violate them bail out — and you get an eslint warning.
It doesn't memoize across mutation	Mutating an object and reusing it still breaks memoization. Use immutable updates.
It doesn't replace algorithmic optimization	An O(n²) filter is still O(n²). Look at the data, not just the renders.
It doesn't help with non-React work	A 200ms blocking request is still 200ms. Web Workers, Suspense, streaming still apply.
It doesn't make a slow render fast	If a single render is slow because the work is heavy, memoization caches it but the first run is the same speed.

The Compiler eslint plugin

Tells you which components opted out of memoization. Treat its warnings as bugs.

```
// .eslintrc.cjs
module.exports = {
  plugins: ['react-compiler'],
  rules: {
    'react-compiler/react-compiler': 'error',
  },
};
```

Common bail-out causes the lint will flag:

- Mutating values inside the render function
- Calling functions with side effects during render
- Using refs.current.value as if it were reactive
- Conditional hook calls (always a violation, regardless of compiler)

When manual memoization still earns its keep

Edge cases the Compiler won't reach. Reach for hand work only when the data says so.

Truly expensive computations
Image processing, large CSV parses — useMemo prevents redundant work even with the Compiler enabled.

Stable references for external libs
Some libraries (D3, charts) misbehave when their props change identity. useCallback gives you that stability.

Components that opted out of the Compiler
Legacy code with side effects in render. eslint plugin shows you which.

Object/array deps in useEffect or useMemo
If you derive an object you pass as a dep, you may need useMemo even with the Compiler — depends on your code shape.

Part 3

Code Splitting

Don't ship code the user won't use. The bundle analyzer is your friend.

React.lazy + Suspense — the basics

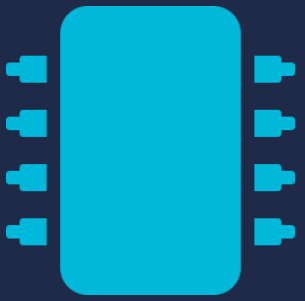
Defer loading a component until it's actually rendered.

```
import { lazy, Suspense } from 'react';

const Settings = lazy(() => import('./Settings'));

function App() {
  const [showSettings, setShowSettings] = useState(false);

  return (
    <>
      <button onClick={() => setShowSettings(true)}>Open settings</button>
      {showSettings && (
        <Suspense fallback={<Spinner />}>
          <Settings />
        </Suspense>
      )}
    </>
  );
}
```

Where code splitting actually wins

Frameworks split routes for free. Your job is to spot the components inside a route.

✅ **Routes — usually automatic**	Next.js and RR v7 split per-route by default. Each page is its own chunk.
✅ **Modal dialogs**	Open by user action, often heavy (rich-text editor, file picker). Lazy-load on first open.
✅ **Admin or settings panels**	Most users never open them. Don't ship that JS in the main bundle.
✅ **Visualization libraries**	Chart.js, D3, Recharts can be 100KB+. Lazy-load the visualization, not the page.
❌ **Tiny utilities**	Splitting a 2KB module costs more in network round-trips than it saves.



Part 4

Beyond the Render

Sometimes the bottleneck isn't React — it's the work React is asking the browser to do.

Virtualization — render only what's visible

Long lists are death. TanStack Virtual renders only the rows in view.

```
import { useVirtualizer } from '@tanstack/react-virtual';

function BigList({ items }) {
  const parentRef = useRef(null);
  const virtualizer = useVirtualizer({
    count: items.length,
    getScrollElement: () => parentRef.current,
    estimateSize: () => 44,
  });

  return (
    <div ref={parentRef} style={{ height: 400, overflow: 'auto' }}>
      <div style={{ height: virtualizer.getTotalSize() }}>
        {virtualizer.getVirtualItems().map(({ key, index, start }) => (
          <Row key={key} data={items[index]} top={start} />
        ))}
      </div>
    </div>
  );
}
```

Web Workers — off the main thread

If a calculation locks the UI, move it to a worker. The main thread stays responsive.

Use a worker when:

- A computation takes >50ms and runs frequently
- Parsing large JSON or CSV blobs
- Image processing in JS (resize, filter)
- Cryptography (hashing, encrypting client-side data)
- Anything that drops INP below 200ms

Don't reach for it when:

- The work is fast (<16ms) — overhead exceeds benefit
- The work needs DOM access (workers can't reach it)
- The data round-trip dominates the work itself
- Server Components could do it instead — usually they should

Comlink (https://github.com/GoogleChromeLabs/comlink) makes worker messaging feel like async function calls. Strongly recommended over raw postMessage.

Throttling vs debouncing

Both limit how often a handler runs. They limit it differently.

Throttle

Run AT MOST every N ms.

- Use for continuous events you want sampled regularly
- Scroll handlers (sticky header, infinite scroll trigger)
- Mousemove handlers (drag tracking)
- Resize handlers

Debounce

Run ONCE after N ms of quiet.

- Use when you only care about the final state
- Search-as-you-type (fire after typing stops)
- Auto-save (after the user stops editing)
- Form validation (after the user pauses input)

Part 5

Putting it Together

A decision flow. AI mistakes. Then the lab.

Performance decision flow

Five questions, in order.

1	**Did you measure?**	*If no — go measure. Profiler + Lighthouse first.*
2	**Is the bundle bloated?**	*Yes → code splitting. React.lazy + Suspense around big modules.*
3	**Is a single component re-rendering too much?**	*Profile says so → check the Compiler eslint output. Manual memo only if needed.*
4	**Is the list long?**	*100+ items → virtualize. TanStack Virtual is the go-to.*
5	**Is a calculation blocking the main thread?**	*Yes → Web Worker. Or move to the server.*

AI mistakes in performance

What to look for when AI suggests an optimization.

- Adding useMemo/useCallback to every value (cargo-culted from 2024)
- React.memo wrapping every component, even ones with complex props
- useEffect for derived data — same mistake as Module 2
- Code splitting tiny utility modules — net regression
- Suggesting Web Workers for fast computations (overhead > work)
- Optimizing without measuring first — the cardinal AI sin here

Module 7 takeaways

What to remember when you're back at your desk.

- Measure before you optimize. Profiler for component cost, Lighthouse for user experience.
- Enable the React Compiler. Most manual useMemo/useCallback/React.memo from 2024 becomes optional.
- Manual memoization still wins for truly expensive work and stable references for external libs.
- Code splitting wins on routes (free), modals, and visualization libraries — not on tiny utilities.
- Virtualize lists with more than ~100 items. Web Workers for >50ms blocking work.
- Throttle for sampled continuous events; debounce for 'final value' events.

Coming up: Lab 6

Profile the social-media app, identify bottlenecks, apply at least three optimizations.

Lab 6: Profile and Optimize

75 minutes · Hands-on · Solution: solutions/lab-06-perf/

What you'll do:

- Run the Profiler and Lighthouse to capture a baseline of the social-media app
- Identify the top three bottlenecks
- Apply at least three different optimizations
- Report before/after numbers (render time, bundle size, or Web Vitals)
- Stretch: add a virtualized posts feed using TanStack Virtual

After the lab: Module 8

Testing React apps in 2026 — Vitest, RTL, MSW, Playwright.

Module 8 covers:

- Vitest as the test runner — fast, ESM-native, Jest-compatible API
- React Testing Library — query priorities, user-event, async patterns
- MSW for API mocking — same mocks for tests and dev
- Playwright for end-to-end (overview)
- Evaluating AI-generated tests — behavior vs implementation

Where we are in Day 3

Performance done. Testing and AI still to come.

Day 3 plan:

- Module 7 — Performance in the Compiler era ✓ (just finished)
- Lab 6 — Profile and optimize (next)
- Module 8 — Testing in 2026
- Lab 7 — Test suite from scratch + AI-generated test review
- Module 9 — Advanced React with AI assistants
- Lab 8 — AI-assisted feature, end to end
- Module 10 — Production concerns + wrap-up

Module 8

Testing React Apps in 2026

Vitest, RTL, MSW, Playwright. And the AI-generated test review skill.

60 minutes · Lecture + demos · Lab 7 follows

The 2026 testing stack

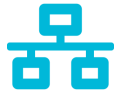
Four tools, each at a different layer.

Vitest

The test runner. Fast, ESM-native, Jest-compatible API. Replaces Jest in new projects.

React Testing Library

The DOM-querying library. Forces you to test what users see, not what your components import.



MSW (Mock Service Worker)

Intercepts network requests. Same mocks for tests and dev. No more mocking fetch by hand.

Playwright

End-to-end. Real browser, real user flows. The expensive layer — use sparingly.

Part 1

Vitest

The Jest-compatible runner for the Vite era.

Why Vitest

Same API as Jest. Faster startup. ESM-native. Plays well with Vite.

Jest-compatible expect, describe, it, vi.fn()	Migrating from Jest is mostly removing config, not rewriting tests.
ESM-native, no babel transform overhead	Test startup measured in hundreds of ms, not seconds.
Watch mode that's actually fast	Re-runs only the affected tests on file change. Real TDD speed.
Built-in coverage via v8 (no Istanbul)	Faster than Jest's coverage. JSON / HTML / lcov outputs supported.
Shares your Vite config	Path aliases, env vars, transformers — all reused from the app's Vite config.

Vitest — setup and a first test

Two files. Done.

```
// vite.config.js
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    setupFiles: ['./src/test/setup.js'],
    globals: true,
  },
});
```

```
// src/sum.test.js
import { sum } from './sum';

describe('sum', () => {
  it('adds two numbers', () => {
    expect(sum(1, 2)).toBe(3);
  });
});

// run: npx vitest
```

Part 2

React Testing Library

Test what users see. Forget about implementation details.

The RTL philosophy

If your test breaks every time you refactor without changing behavior, your test is wrong.

> *"The more your tests resemble the way your software is used, the more confidence they can give you."*
>
> *— Kent C. Dodds, RTL author*

Practical implications

- Query by ROLE first, then by accessible name, label, or text — not by class names or test IDs
- Simulate USER actions (user-event), not DOM events (fireEvent)
- Assert on what the user can perceive — visible text, focused element, navigation
- Don't test that a useState was called or that a callback received a specific argument

Query priority hierarchy

Use the highest-priority query that fits. The lower you go, the brittler the test.

1	`getByRole`	Accessible to everyone, including screen readers. Almost always the right answer.
2	`getByLabelText`	Form fields. Labels carry semantic meaning.
3	`getByPlaceholderText`	Fallback for inputs without a proper label. Worse than 2 — fix the label instead.
4	`getByText`	Non-interactive content. Use sparingly — text changes more than roles.
5	`getByDisplayValue`	The current value of an input. Useful in form-driven tests.
6	`getByAltText, getByTitle`	Images, abbreviations. Tied to accessible attributes.
7	`getByTestId`	Last resort. Treat as code smell — usually means the UI lacks accessibility metadata.

user-event vs fireEvent

Both type characters into a field. Only one simulates what the user actually does.

fireEvent — low-level

```
import { fireEvent } from '@testing-library/react';

fireEvent.change(input, {
  target: { value: 'hello' },
});
// fires one synthetic event
```

Skips: focus, keypress, key validation, paste handling.

user-event — high-level

```
import userEvent from '@testing-library/user-event';

const user = userEvent.setup();
await user.type(input, 'hello');
// fires focus, keydown, keypress, keyup,
// input, change — for each character
```

Catches: missing key handlers, paste handlers, IME input, more.

Default to user-event. Reach for fireEvent only when user-event can't simulate the action (rare in practice).

Async patterns — find vs waitFor

Most modern UI is async. Use the right primitive for the wait.

```
// findBy: returns a promise that resolves when an element appears
const post = await screen.findByRole('article', { name: /welcome/i });

// waitFor: re-runs the assertion until it passes (or times out)
await waitFor(() => {
  expect(screen.getByText(/loading/i)).not.toBeInTheDocument();
});

// waitForElementToBeRemoved: when something disappears
await waitForElementToBeRemoved(() => screen.queryByText(/loading/i));

// AVOID: arbitrary timeouts
await new Promise(r => setTimeout(r, 1000)); // flaky
```

A complete RTL example

Test the form's behavior, not its internals.

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import LoginForm from './LoginForm';

describe('LoginForm', () => {
  it('shows a validation error when the password is empty', async () => {
    const user = userEvent.setup();
    render(<LoginForm />);

    await user.type(screen.getByLabelText(/email/i), 'a@b.com');
    await user.click(screen.getByRole('button', { name: /sign in/i }));

    expect(await screen.findByText(/password is required/i))
      .toBeInTheDocument();
  });
});
```

Part 3

MSW (Mock Service Worker)

Mock at the network layer. Same mocks for tests AND dev.

Why MSW

Mocking fetch by hand has tradeoffs MSW avoids.

Without MSW

- vi.fn() per fetch call — easy to forget one
- Mocks live in tests; dev still hits the real API
- Refactoring the API layer breaks all tests
- Hard to simulate retries or error sequences

With MSW

- Mocks intercept network requests below fetch
- Same handlers run in tests, dev (service worker), and Storybook
- Refactoring api.js doesn't break tests
- Easy to simulate sequences: error then success on retry

MSW intercepts at the network layer using the Service Worker API in dev and a Node-side interceptor in tests. Your application code calls fetch as normal — it never knows about the mock.

MSW — setup and handlers

Define handlers once. Use them everywhere.

```
// src/test/handlers.js
import { http, HttpResponse } from 'msw';

export const handlers = [
  http.get('/api/posts', () => {
    return HttpResponse.json([
      { id: 1, title: 'Hello', body: '…' },
    ]);
  }),
  http.post('/api/posts', async ({ request }) => {
    const post = await request.json();
    return HttpResponse.json({ id: 2, ...post }, { status: 201 });
  }),
  http.get('/api/posts/:id', ({ params }) => {
    return HttpResponse.json({ id: +params.id, title: '…' });
  }),
];
```

MSW in tests

Wire it up once in test setup. Override handlers per-test for edge cases.

```
// src/test/setup.js
import { setupServer } from 'msw/node';
import { handlers } from './handlers';

const server = setupServer(...handlers);

beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());
afterAll(() => server.close());

// Per-test override:
it('shows error on 500', async () => {
  server.use(
    http.get('/api/posts', () => HttpResponse.error())
  );
  render(<PostsFeed />);
  expect(await screen.findByText(/something went wrong/i)).toBeInTheDocument();
});
```

Part 4

Playwright

End-to-end tests in a real browser. The expensive layer.

When E2E is the right tool

Slow. Expensive to write. Highest confidence. Use sparingly.

Reach for Playwright when:

- Testing critical user flows end-to-end (auth, checkout, payment)
- Catching cross-browser bugs (Safari, Firefox, Chrome)
- Testing the real backend, not a mock
- Visual regression — screenshots over time
- Smoke tests against staging or production

Don't reach for Playwright when:

- Testing a single component's rendering (use RTL)
- Testing edge cases that need many runs (slow)
- You don't have CI capacity for browser test runs
- Your team won't maintain flaky tests (E2E is flaky)

Rule of thumb: 80% RTL/unit, 15% integration, 5% E2E. Inverted ratios mean expensive, slow, flaky test suites.

Playwright at a glance

Real browser, real DOM, codegen for the boilerplate.

```ts
// e2e/login.spec.ts
import { test, expect } from '@playwright/test';

test('user can sign in and reach the dashboard', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('a@b.com');
  await page.getByLabel('Password').fill('secret123');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page).toHaveURL(/.*dashboard/);
  await expect(page.getByRole('heading', { name: /welcome/i })).toBeVisible();
});

// run: npx playwright test
// or: npx playwright codegen http://localhost:5173 (records actions)
```

Part 5

Testing the Server Side

The newest territory. Tools are still maturing.

Testing Server Components and Actions

The story is incomplete in mid-2026. Here's where it stands.

Server Action functions test like any async function
Import the action, call it with FormData, assert on the return value or DB state. Standard Vitest.

Server Components — mostly E2E for now
RTL doesn't render Server Components directly. Either test the data layer with unit tests + the rendered output with Playwright, or use Next.js's experimental test helpers.

Mock the data layer, not the framework
Test that getPosts() returns the expected shape; test that the page renders that shape correctly. Don't try to mock React's renderer.

When in doubt, push to the boundary
Server Action contracts (input/output) and Client Component renders are both well-understood. The seam between them is where E2E earns its keep.

Part 6

AI-Generated Tests

AI writes a lot of tests. Most of them test the wrong thing.

Behavior vs implementation

Two tests for the same component. One survives a refactor; one doesn't.

Implementation test (don't)

```
it('sets isOpen state on click', () => {
  const setState = vi.spyOn(React, 'useState');
  render(<Drawer />);
  fireEvent.click(screen.getByTestId('open-btn'));
  expect(setState).toHaveBeenCalledWith(true);
});

// breaks if you switch to useReducer
// breaks if you rename the state var
// breaks if you change the test ID
```

Behavior test (do)

```
it('opens drawer on click', async () => {
  const user = userEvent.setup();
  render(<Drawer />);

  const btn = screen.getByRole('button');
  await user.click(btn);

  expect(screen.getByRole('navigation'))
    .toBeVisible();
});

// passes for useState, useReducer,
// Zustand, or RTK — same behavior
```

AI test red flags

When you read AI-generated tests, look for these.

- vi.spyOn() on React's hooks — testing internals
- getByTestId everywhere instead of getByRole
- expect(setState).toHaveBeenCalledWith(...) — implementation coupling
- fireEvent instead of userEvent — misses real user interactions
- Mocking individual functions in api.js — should mock at the network
- setTimeout-based waits — flakiness factory
- Tests that pass without rendering anything — checking pure utils only
- Snapshot tests for entire components — every refactor fails them

Module 8 takeaways

What to remember when you're back at your desk.

- Vitest > Jest for new projects. Same API, faster startup, shared config with Vite.
- Test BEHAVIOR, not implementation. Query by ROLE first; getByTestId is the last resort.
- user-event over fireEvent. Always await it. It simulates real interaction.
- MSW intercepts at the network layer. Same mocks for tests AND dev. Reset handlers between tests.
- Playwright for critical flows only. 80/15/5 — unit/integration/E2E.
- AI-generated tests need review. Look for spyOn, fireEvent, getByTestId, setTimeout.

Coming up: Lab 7

Write a baseline test suite, then have AI extend it. Review every line.

Lab 7: Test Suite from Scratch

75 minutes · Hands-on · AI-test review is the load-bearing part

What you'll do:

- Set up Vitest + RTL + MSW for the social-media app
- Write a baseline test suite for one feature by hand
- Have AI extend the suite
- Mark each AI-generated test as keep / refactor / delete with a written reason
- Stretch: add one Playwright e2e test for the auth flow

After the lab: Module 9

Advanced React with AI assistants — the focused module.

Module 9 brings together the AI thread that's run through every day:

- Tool-agnostic patterns that work across Cursor, Claude Code, Copilot, Windsurf
- Taxonomy of React mistakes LLMs make — pulled from your own findings this week
- Project context for AI: AGENTS.md / CLAUDE.md, lint rules, repo conventions
- AI-assisted refactors and migrations — and verification
- Lab 8: build a feature end-to-end with AI assistance, then code-review yourself

Day 3 progress

Three modules done. AI module + final wrap-up still ahead.

Day 3 plan:

- Module 7 — Performance in the Compiler era ✓
- Lab 6 — Profile and optimize ✓
- Module 8 — Testing in 2026 ✓ (just finished)
- Lab 7 — Test suite + AI-generated test review (next)
- Module 9 — Working with AI assistants
- Lab 8 — AI-assisted feature, end to end
- Module 10 — Production concerns + course wrap-up

Module 9

Advanced React with AI Assistants

Tool-agnostic patterns. The mistake taxonomy. Project context. The review checklist.

75 minutes · Lecture + demos · Lab 8 follows

The thread so far

You've spotted AI-generated React mistakes in every lab this week.

Lab 1	Stale closures, useEffect for derived state, race-prone setters
Lab 2	v6 React Router patterns, Pages-Router-era Next.js, localStorage for sessions
Lab 3	Server data in client store, setter-shaped action names
Lab 4	Mutable query keys, missing invalidation, retry on mutations
Lab 5	'use client' as escape hatch, server-only imports leaking, non-serializable props
Lab 6	Optimize without measure, useMemo everywhere, Web Worker overhead
Lab 7	getByTestId, fireEvent, snapshot tests, spyOn React.useState

Part 1

The AI Tool Landscape

Four big tools, one principle: agnostic patterns over tool-specific tricks.

Tool-agnostic principles

Any tool gets better when you apply these. None gets bad when you don't.

Read every diff	Don't accept output without reading. The friction of reading is what catches the bugs. AI gets faster — your reading discipline shouldn't.
Give it project context	AGENTS.md, CLAUDE.md, lint rules, conventions — all of these scope the AI's output to your codebase's reality.
Specify the contract, not the steps	"This function takes X, returns Y, mutates nothing" beats "first do A, then B, then C". The AI is good at filling in the middle.
Iterate small	One hook, one component, one feature. Big batches mean big diffs you can't review. Small batches give the AI feedback in the loop.
Verify with running code, not just reading	Tests, type checks, the dev server. Reading catches some mistakes; running catches the rest.

Four tools, different shapes

Pick by workflow, not by hype. Most teams settle on one or two.

Cursor

VS Code fork. Inline edits + agent panel. Diff-driven. Strong for editor-in-the-loop work.

Claude Code

CLI agent. Reads/edits files, runs commands. Strong for long autonomous tasks.

Copilot

GitHub-integrated. In-editor completions + chat. Strong for 'autocomplete on steroids'.

Windsurf

IDE with agent + Cascade flow. Good middle ground between Cursor and CLI agents.

All four use similar underlying models. The differences are workflow surface, not capability ceiling. Pick what fits how you actually work.

Editor-in-the-loop vs agent-in-the-loop

Two workflow modes. They suit different tasks.

Editor-in-the-loop

You drive; AI suggests.

- Inline completions + small edits
- Tight loop — read, accept/reject, continue
- Best for: well-understood code, refactors with clear intent, single-file changes
- Tools: Cursor, Copilot, IDE chat panels

Agent-in-the-loop

You spec; AI executes.

- Multi-file changes, tool use, autonomous loops
- Loop is longer — minutes, not seconds
- Best for: greenfield features, repetitive edits across many files, migrations
- Tools: Claude Code, Cursor agent mode, Windsurf Cascade

Part 2

Taxonomy of Mistakes

Six categories. The same patterns across every tool.

Six recurring mistake categories

The same six show up regardless of which AI tool you use.

1	**Stale closures and missing deps**	Effects that capture old values. Always the top of the list.
2	**useEffect for things that aren't effects**	Derived state, prop synchronization, event-triggered actions.
3	**Server/client boundary violations**	'use client' as escape hatch, server imports leaking, non-serializable props.
4	**Hydration mismatches**	Date.now(), Math.random(), navigator checks during render.
5	**Outdated patterns from training data**	v6 React Router, Pages-Router Next.js, class components, Redux without Toolkit.
6	**Test coupling and accessibility skips**	getByTestId, fireEvent, snapshot tests, spying on hooks.

1. Stale closures and missing deps

The most common AI-generated React bug. Eslint catches most; some slip through.

Anti-pattern

```
useEffect(() => {
  const id = setInterval(() => {
    console.log(count);  // always 0
  }, 1000);
  return () => clearInterval(id);
}, []);  // missing [count]
```

Fix

```
useEffect(() => {
  const id = setInterval(() => {
    setCount(n => n + 1);
  }, 1000);
  return () => clearInterval(id);
}, []);  // safe — functional updater
```

Eslint plugin react-hooks/exhaustive-deps catches the missing-dep version. Don't disable it. Functional setState avoids the dep entirely.

2. useEffect for things that aren't effects

If you can derive it, don't sync it. If it's an event response, don't put it in an effect.

Derived state via useEffect + useState

filteredItems = items.filter() should be inline, not synchronized via setState.

useEffect to reset state when prop changes

Pass a key prop. React unmounts + remounts the subtree.

useEffect to fetch on mount

Use Suspense + use(), TanStack Query, or a route loader. Module 5.

useEffect to dispatch on user action

Move the dispatch into the event handler. Not every state change is an effect trigger.

3. Server/client boundary violations

The 2026-specific category. Module 6's content, in field form.

- 'use client' added at the top of a Server Component to silence a hook error
- Server-only utility (e.g., db client) imported into a 'use client' file
- Inline function passed as a prop from Server to Client Component (must be Server Action)
- useState in a Server Component (use a Client Component for state)
- Server Action without 'use server' directive — runs as a regular client function
- Reading cookies() or headers() inside a Client Component

4. Hydration mismatches

Server renders one thing, client renders another. Often non-deterministic.

Date.now() in render
Server time != client time. Use a stable date or render only on client.

Math.random() in render
Different on each render. Pre-compute or move to useState init.

navigator.* / window.* checks
Undefined on server. Wrap in useEffect or use 'use client'.

Locale-dependent formatting
Server locale != browser locale. Use a stable formatter or render client-side.

5. Outdated patterns from training data

AI learned React from years of code. Some of that code is from 2020.

Old pattern	Modern
v6 BrowserRouter / Routes / Route JSX	→ RR v7 routes config + framework mode
Pages Router (getServerSideProps)	→ App Router (Server Components + Actions)
Class components	→ Function components + hooks
Redux with createStore + Thunk	→ Redux Toolkit (or skip Redux entirely)
componentDidMount / componentWillUnmount	→ useEffect with cleanup
Manual useMemo/useCallback everywhere	→ React Compiler (most cases)

6. Test coupling and accessibility skips

Module 8's red-flag list, in field form.

- getByTestId everywhere instead of role-based queries
- fireEvent instead of userEvent (misses real interactions)
- vi.spyOn on React hooks — testing internals
- Snapshot tests for entire components
- Mocking individual api.js functions instead of network mocks
- setTimeout-based waits — flakiness factory

Part 3

Project Context

Set the rules in writing. AI plays by them.

AGENTS.md / CLAUDE.md

A markdown file at your repo root. Tells AI tools how your project does things.

What it does

- Documents conventions: file naming, where state lives, how routes are structured
- Lists 'do not' rules: don't use useEffect for derived state, don't add deps without asking
- Specifies test commands: 'run tests with npm test'
- Names key files the AI should always read first

Why it works

- Most AI tools auto-load AGENTS.md (or CLAUDE.md, depending on tool)
- Overrides training-data drift — 'we use RR v7' beats whatever the AI learned
- Lets new contributors and AI agents share the same context
- Versioned in git — conventions evolve with the codebase

AGENTS.md is the emerging cross-tool standard. CLAUDE.md is Claude-specific. Both can exist in a repo — most tools read whichever they expect.

Example AGENTS.md

Short, opinionated, code-grounded. Avoid abstractions; name files.

```
# AGENTS.md — social-media-rr-v7

## Stack
React 19, React Router v7 (framework mode), TanStack Query, Zustand for client state.
Vite 6. Vitest + RTL + MSW for tests. ESLint with react-hooks and react-compiler plugins.

## Conventions
- All routes go in app/routes/. Loaders + actions are exports from those files.
- Server data goes through TanStack Query. Don't use useEffect + fetch.
- Tests use userEvent.setup(), getByRole, MSW handlers (NOT vi.fn on api.js).
- Action names describe transitions: addItem, removePost. Not setItems, setPosts.

## Do not
- Add 'use client' to silence build errors. Refactor instead.
- Use getByTestId in tests. Use accessible queries.
- Add localStorage for auth tokens. Cookies are HttpOnly server-managed.

## Read first
- app/lib/queryKeys.js (cache key conventions)
- app/stores/ui.js (theme + sidebar — example of our store style)
```

Lint rules as guardrails

AI listens to errors. Make the lint config strict so AI fixes things on the way out.

`react-hooks/exhaustive-deps`	Catches missing useEffect deps. Catches stale closures the AI introduces.
`react-compiler/react-compiler`	Flags components that bail out of compiler memoization. Catches in-render mutation.
`@typescript-eslint/no-floating-promises`	If TS: catches AI's tendency to forget await.
`testing-library/prefer-user-event`	Auto-flags fireEvent suggestions in test files.
`no-restricted-imports (server-only / client-only)`	Prevents server modules leaking into client and vice versa.

Repo conventions that travel

Small, codified, durable. AI tools across your team see the same shape.

`queryKeys.js`	Centralize TanStack Query keys. AI uses your factories instead of inventing new ones.
`stores/`	Group client state by feature. AI mirrors the structure when adding state.
`actions/`	Server Actions in one folder. 'use server' at the top of each file makes the boundary obvious.
`lib/api.js`	Single source of network calls. AI extends it instead of rolling new fetch patterns.
`test/handlers.js`	MSW handlers in one place. AI extends, doesn't fragment, the mock surface.

Part 4

Workflows That Work

Spec-driven dev. AI-assisted refactors. AI-assisted migrations.

Spec-driven development

Write a brief spec. Let the AI do the typing. Review the diff.

1	**Write a one-paragraph spec**	What does the feature do? What's the acceptance criteria? Where does the code go? What's out of scope?
2	**Drop the spec into the AI's context**	Either as a prompt or as a SPEC.md file the agent reads.
3	**Let the AI propose a structure first**	File list, function signatures, type definitions. Review BEFORE implementation.
4	**Iterate small**	One file at a time. Read each diff. Run tests after each batch.
5	**Verify with tests, types, dev server**	All three layers. Bugs that survive any one layer often die in another.

AI-assisted refactors

AI is great at mechanical refactors. Verify with tests, not vibes.

Class to function components
Lab 1's pattern. AI handles the syntactic conversion; you fix the semantic mistakes (stale closures, state-in-effect).

Old hooks to new hooks
useState to useReducer, useEffect+fetch to TanStack Query. AI gets the structure right; you check the dep arrays.

Renaming across files
AI is faster than your IDE for cross-file renames with logic changes. Verify with tests.

API shape changes
Backend renames a field; frontend has 30 references. AI finds them; you spot-check the edge cases.

AI-assisted migrations

Bigger than refactors. Plan more deliberately.

- Lock down a known-good baseline first — tests pass, types check, dev server works
- Migrate one module/route/feature at a time, not the whole repo
- Run the full test suite after each batch — if anything breaks, revert and re-prompt
- Commit each migrated module as its own change — one change, one revert path
- Keep a CHANGELOG of what was migrated, what's still old, and what each commit covered
- When in doubt, ask the AI to explain its choices BEFORE you accept the diff

Part 5

The Review Checklist

One page. Carry it into every review of AI-generated React.

AI-generated React review checklist

Carry it. Apply it. Lab 8 puts it into practice.

Hooks

- Stale closures in useEffect?
- Missing dep array entries?
- Hooks inside conditionals or loops?
- useEffect for derived state or events?

Patterns

- Modern Router APIs (RR v7 / Next.js App Router)?
- Modern state (Zustand/Jotai/RTK or just useState)?
- Server data through TanStack Query, not useEffect+fetch?
- Functional setState where it matters?

Boundaries

- 'use client' at the right level?
- Server-only imports staying server-side?
- Functions across boundary marked 'use server'?
- Non-serializable props eliminated?

Tests

- Role-based queries, not getByTestId?
- user-event, not fireEvent?
- MSW for network mocks, not vi.fn on api?
- No spies on React internals?

Module 9 takeaways

What to remember when you're back at your desk.

- Read every diff. The friction of reading is what catches the bugs.
- Six recurring AI mistakes: stale closures, useEffect overreach, boundary violations, hydration mismatches, outdated patterns, test coupling.
- AGENTS.md / CLAUDE.md is your project context for AI. Short, opinionated, names files.
- Lint rules are guardrails. Don't disable to let AI's code compile.
- Spec → structure → small batches → verify with tests + types + running code.
- Carry the review checklist. Roles, hooks, boundaries, patterns, tests.

Coming up: Lab 8

Build a feature end-to-end with AI. Self-review the result.

Lab 8: AI-Assisted Feature, End to End

90 minutes · Hands-on · Self-review is the deliverable

What you'll do:

- Pick one feature from a provided backlog (or propose your own)
- Write a brief spec (one paragraph)
- Build the feature with heavy AI assistance
- Self-review using the checklist — find at least three issues you fixed
- End with working code + a written self-review listing the issues you caught

After the lab: Module 10

Production concerns + course wrap-up.

Module 10 covers:

- React-specific security: CSP for streaming SSR, secrets in Server Actions, hydration safety
- Accessibility — what AI assistants frequently miss
- Internationalization briefly
- Course wrap-up, Q&A, where to go next

Day 3 progress

Almost there.

- Module 7 — Performance ✓
- Lab 6 — Profile and optimize ✓
- Module 8 — Testing in 2026 ✓
- Lab 7 — Test suite + AI review ✓
- Module 9 — AI assistants ✓ (just finished)
- Lab 8 — AI-assisted feature end-to-end (next)
- Module 10 — Production + wrap-up

Module 10

Production Concerns + Wrap-up

Security, accessibility, i18n, and where to go from here.

45 minutes · Lecture + Q&A · Final module

Part 1

React-Specific Security

Generic security advice + the things that bite React apps in particular.

dangerouslySetInnerHTML — when and how

React escapes content by default. The exception is the danger zone.

When you actually need it

- Rendering pre-sanitized HTML from a CMS
- Inlining markdown that's already been parsed and sanitized
- Embedding SVG content as a string
- Inlining critical CSS for first paint (rare)

Sanitize first, render second

```
import DOMPurify from 'isomorphic-dompurify';

<div
  dangerouslySetInnerHTML={{
    __html: DOMPurify.sanitize(html)
  }}
/>
```

AI tools sometimes use dangerouslySetInnerHTML to render plain user input — that's an XSS bug. Always sanitize, or use plain children if the content is text.

CSP for streaming SSR

Streaming responses make the inline-script story trickier than 2024.

Inline scripts in streaming SSR need a nonce
React injects scripts during streaming hydration. Without a nonce in the CSP, browser blocks them — silently breaking your app.

Use a per-request nonce
Generate at request time, attach to the response's CSP header AND pass to React's renderToPipeableStream as the nonce option.

Frameworks handle this for you
Next.js 15 and RR v7 framework mode generate nonces automatically. Vanilla SSR setups need to wire it manually.

Don't use 'unsafe-inline' to make it work
Defeats the entire CSP. The five minutes to set up a nonce save you the actual XSS bugs CSP exists to prevent.

Secrets in Server Actions

Server Actions run on the server. React encrypts closure-captured values server-side, but reading secrets INSIDE the action is still better hygiene — clearer intent, cleaner reviews, less surface area for accidents.

Closure-captured (works, but…)

```
// page.jsx (Server Component)
const apiKey = process.env.SECRET_KEY;

async function doThing(formData) {
  'use server';
  return fetch(url, {
    headers: { Authorization: apiKey }
  });
}

// apiKey is captured in the closure
// → React encrypts it server-side
// → only an opaque action ID reaches the client
```

Read inside the action (preferred)

```
async function doThing(formData) {
  'use server';
  // read inside the action
  const apiKey = process.env.SECRET_KEY;
  return fetch(url, {
    headers: { Authorization: apiKey }
  });
}

// apiKey read at call time on the server
// → never enters a closure; trivially auditable
```

Auth tokens belong in HttpOnly cookies

Not localStorage. Not sessionStorage. Not 'just for now.'

localStorage
Readable by any JS on the page. One XSS = full account takeover. Don't use for tokens, ever.

sessionStorage
Same XSS exposure. The 'session-scoped' part doesn't help — XSS happens during the session.

Plain cookies (not HttpOnly)
Still readable by JS. Only marginally better than localStorage.

HttpOnly + Secure + SameSite cookies
Set by the server. Sent automatically on requests. Not accessible to JavaScript. The right answer.

Part 2

Accessibility

What AI assistants miss. What you should always check.

The accessibility minimum bar

Six things that aren't optional. AI gets some right; gets others embarrassingly wrong.

- Every interactive element is a button, link, or properly-labeled input
- All form fields have a visible <label> with htmlFor + id wiring
- Color contrast meets WCAG AA (4.5:1 for body, 3:1 for large text)
- Focus states are visible — never outline: none without a replacement
- Modal dialogs trap focus; closing returns focus to the opener
- All non-decorative images have alt text; decorative ones have alt=""

What AI tools systematically miss

These are the ones that show up in code review unless you're looking.

- <div onClick> for buttons (no keyboard handler, no role, no focus)
- Placeholder used as label (disappears when typing, screen reader skips it)
- Custom dropdown without keyboard navigation (arrow keys, Escape)
- Modal that doesn't trap focus or return it on close
- Loading spinner without aria-live='polite' or aria-busy
- Error messages not associated with the failing field via aria-describedby
- Icon buttons without aria-label (just an emoji or SVG inside <button>)
- Tooltips on hover only (unreachable by keyboard)

Accessibility review tools

Three tools that catch most issues. None catches all.

eslint-plugin-jsx-a11y

Static checks. Catches missing alt, onClick on non-interactive elements, label associations. Add to your eslint config — it's lightweight.

axe DevTools (browser extension)

Runtime audit. Tests color contrast, ARIA roles, keyboard nav, focus management. Run on every page before shipping.

Manual keyboard testing

Tab through the page. Can you reach every interactive element? Can you escape modals? If not, neither can keyboard-only users.

Part 3

Internationalization

Briefly. Enough to know what to reach for.

The minimum viable i18n

Three decisions, one library, done.

1

Pick a library

react-intl (mature, FormatJS-based) or next-intl (Next.js-flavored). Both wrap the Intl Web API and give you a t() function.

2

Externalize strings

Move every user-visible string into translation files. Avoid string concatenation — use ICU MessageFormat for plurals, gender, dates.

3

Decide where the locale lives

URL prefix (/en/, /fr/) for SEO. Cookie-based for app state. Don't sniff Accept-Language and forget it — make the choice user-controllable.

Add i18n early if you might need it. Retrofitting is painful — string concatenation spreads across the codebase.

Part 4

Course Wrap-up

Three days. Ten modules. Eight labs. Where to go next.

What we covered

Three days, ten modules, eight labs.

Day 1 — Foundations

Modules: Landscape · Hooks · Routing

Labs: Modernize chat · Routing in two frameworks

Day 2 — Data, State, Server

Modules: State · Data fetching · Server Components

Labs: Pick-your-tool refactor · TanStack Query · RSC dashboard

Day 3 — Performance, Quality, AI

Modules: Performance · Testing · AI · Production

Labs: Profile + optimize · Test suite + AI review · AI-assisted feature

Ten things to take back to work

If you remember nothing else, remember these.

1. Read every diff. Especially AI-generated ones.
2. useEffect synchronizes with external systems. Almost everything else is a misuse.
3. Server state belongs in TanStack Query. Client state belongs in useState (or a small store).
4. RR v7 framework mode and Next.js App Router solve the same problems. Pick by team and deploy story.
5. Server Components run on the server. 'use client' is opt-in, not the default.
6. The React Compiler eliminates most manual memoization. Enable it and the eslint plugin.
7. Measure before you optimize. Profiler + Lighthouse + bundle analyzer.
8. Test behavior, not implementation. getByRole, userEvent, MSW.
9. AGENTS.md / CLAUDE.md is your project context for AI. Short, specific, names files.
10. HttpOnly cookies for auth. Never localStorage.

Where to go next

Concrete next steps for the week after the course.

This week: pick one thing

Enable the React Compiler in one project. Or migrate one route to Server Components. Or write AGENTS.md for one repo. Don't try all three.

Next month: deepen one area

If RSC bent your mind, build something with Next.js or RR v7 framework mode. If state shape clicked, refactor a real codebase. Pick the one that matters most for your work.

Make the AI thread a habit

Pull the Module 9 checklist into your team's PR template. Write your team's AGENTS.md. Establish review norms for AI-generated code.

Share what worked

Internal lunch-and-learn on one module. The teaching is the test of whether you understood it.

Resources for continued learning

Links that will still be relevant in six months.

`react.dev`	The official React docs. The hooks reference is the canonical source.
`Next.js docs (nextjs.org)`	Best place to deepen RSC and Server Actions practice.
`TanStack docs (tanstack.com)`	Query, Router, Table, Virtual — all share a sensibility worth learning.
`react-router.com`	Framework mode is well-documented; the migration guide from v6 is solid.
`Kent C. Dodds blog (kentcdodds.com)`	RTL, hook patterns, common mistakes. Especially the testing posts.
`react.dev Blog`	Where the React team announces the things you'll use next year.

Keep the lab repo

It's yours to use. Reference, fork, extend.

github.com/chrisminnick/advanced-react-19

What's in it:

- All starters from this week, under lab-files/lab-01 through lab-08
- Reference solutions for every lab, under solutions/lab-NN-*
- AGENTS.md / CLAUDE.md at the repo root — project context for AI assistants
- Backlog of feature ideas (the Lab 8 backlog and more)

Questions

Anything from this week. Anything from your work. Open floor.

Some prompts if you're not sure where to start:

- What's the biggest blocker for adopting React 19 at your company?
- Where would Server Components save the most time in your current app?
- Which AI mistake category bites your team most often?
- What's the one decision from this week you're least sure about?

Thank you

for spending three days with me.

Chris Minnick
chris@minnick.com
watzthis.com

Course feedback

If you have a minute — what worked, what didn't, what would you change?

Three things to think about as you fill in feedback:

- Pace — too fast, too slow, or about right?
- Lab time — was 50% the right balance? More? Less?
- AI thread — useful, distracting, or about right?
- Anything you wish we'd covered?
- Anything we covered too much?